

FINAL-YEAR B.TECH AI & ML MAJOR CAPSTONE PROJECT REPORT

GoTicket

AI-Agent-Based Intercity Bus Booking Platform

Project Title:	GoTicket -- AI-Agent-Based Intercity Bus Booking Platform
Degree & Branch:	Bachelor of Technology (B.Tech) in AI & ML / CSE
Project Nature:	Major Final-Year Capstone Group Project (4 Members)
Academic Year:	2025 - 2026
Department:	Department of Artificial Intelligence & Machine Learning
Student 1 (AI & NLU Lead):	Anshika Pandey (2k23.psitaiml2311222@gmail.com)
Student 2 (Frontend UI Lead):	Jaishree Verma (jaishree.verma.work@gmail.com)
Student 3 (Booking Engine Lead):	[Student 3 Name Placeholder]
Student 4 (Telemetry & Auth Lead):	[Student 4 Name Placeholder]
Project Supervisor / Guide:	[Project Guide Name & Designation Placeholder]
Repository URL:	https://github.com/jaishree-verma/Go-Ticket.git

PART 1: MASTER PROJECT REPORT

GoTicket — AI-Agent-Based Intercity Bus Booking Platform

Major B.Tech Final Year Project Report

Department of Computer Science & Engineering / Artificial Intelligence & Machine Learning

Academic Year: 2025–2026

Project & Group Identification

Attribute	Project Details
Project Title	GoTicket — AI-Agent-Based Intercity Bus Booking Platform
Degree & Branch	Bachelor of Technology (B.Tech) in AI & ML / CSE
Project Nature	Major Final-Year Capstone Group Project (4 Members)
Repository URL	https://github.com/jaishree-verma/Go-Ticket.git
Target Branch	<code>master</code> / <code>main</code>
Application Architecture	Single Page Application (SPA) with Decoupled Service Layer
Frontend Framework	React 19 (<code>react</code> 19.1.1, <code>react-dom</code> 19.1.1, <code>react-router-dom</code> 7.8.2)
Styling Paradigm	Component-Scoped CSS Modules + Global Theme Tokens
State & Persistence	React Context API + Browser <code>localStorage</code>
Deployment Platform	Vercel Serverless Edge Hosting (<code>vercel.json</code>)

Group Members

1. Student 1 (AI / Conversational Agent Lead):

* Name: Anshika Pandey * Roll Number: [Roll Number Placeholder 1] * Email: 2k23.psitauml2311222@gmail.com
2. Student 2 (Frontend Architecture & UI Lead):

* Name: Jaishree Verma * Roll Number: [Roll Number Placeholder 2] * Email: jaishree.verma.work@gmail.com
3. Student 3 (Booking Engine & Checkout Lead):

* Name: [Student 3 Name Placeholder] * Roll Number: [Roll Number Placeholder 3] * Email: [Student 3 Email Placeholder]
4. Student 4 (Telemetry, Authentication & Integration Lead):

* Name: [Student 4 Name Placeholder] * Roll Number: [Roll Number Placeholder 4] * Email: [Student 4 Email Placeholder]

Academic Supervision

- Project Guide / Supervisor: [Project Guide Name & Designation Placeholder]
- Department: Department of Artificial Intelligence & Machine Learning
- College / Institution: [College / Institute Name Placeholder]

1. Abstract

Digital bus ticket reservation in the Indian intercity transit sector is traditionally characterized by complex, form-heavy interfaces that present high cognitive load, require repetitive manual filtering across disparate parameters, and lack contextual intelligence. Travelers are forced to manually parse through dozens of bus schedules, compare varying fares and timings, navigate rigid multi-step forms, and manage disjointed post-booking communications.

GoTicket resolves these operational inefficiencies by introducing a unified, dual-mode intercity bus reservation platform. GoTicket merges a full-featured **Manual Booking Web Workflow** with an autonomous, multi-turn conversational AI travel concierge named **"Tixie"**. Tixie is engineered using a deterministic client-side Natural Language Understanding (NLU) parser, a finite state machine (FSM), a 5-factor multi-criteria bus recommendation engine, and an atomic seat validation module.

Through Tixie, commuters can express travel requirements in free-form natural language (e.g., *"Find buses from Kanpur to Delhi tomorrow evening"*), receive deterministically ranked bus recommendations based on a 5-factor scoring model (Time Match: 30%, Price: 25%, Duration: 20%, Availability: 15%, Operator Rating: 10%), select seats atomically to prevent fragmented allocations, submit passenger details in a single sentence or step-by-step, inspect an itemized booking summary, and proceed through a strict confirmation gate prior to payment handoff. Simultaneously, GoTicket delivers a streamlined manual booking interface featuring 50+ city auto-suggestions, an interactive 40-seat bus layout with occupied seat locks, boarding/dropping point selectors, multi-passenger checkouts with simulated 6-digit mobile OTP verification, promotional coupon discounts, printable digital E-Tickets with QR verification codes, and simulated live vehicle telemetry on OpenStreetMap with Haversine distance estimation.

Academic & Technical Disclosure: GoTicket is implemented as a self-contained, client-side Single Page Application (SPA) prototype. Application state, NLU processing, recommendation algorithms, atomic seat validations, and mock services run within client-side JavaScript services with data persistence in browser `localStorage`. External systems—including physical GPS hardware, production payment gateways, SMS/email SMTP gateways, persistent SQL databases, and cloud LLM APIs—are simulated for robust, dependency-free demonstration and evaluation.

2. Introduction

2.1 Background & Industry Overview

Intercity road passenger transportation represents one of the largest mobility segments in India, serving millions of commuters daily across high-density corridors such as Kanpur–Delhi, Lucknow–Kanpur, and Delhi–Agra. Despite widespread smartphone penetration, online bus reservation interfaces frequently remain cluttered, relying on static tabular displays and rigid form selectors that create friction for mobile users and non-technical travelers.

2.2 The Paradigm Shift to Conversational Commerce

Conversational interfaces provide a natural, human-centric alternative to traditional graphical user interfaces. Rather than requiring users to manually fill out source, destination, calendar dates, departure filters, and operator preferences across nested web forms, an intelligent conversational agent can extract intent, normalize travel parameters, resolve ambiguities, and execute domain actions through goal-oriented dialogue.

2.3 Motivation Behind GoTicket

The GoTicket project was conceived by a 4-member final-year B.Tech AI & ML team to explore how modern web engineering and deterministic AI algorithms can converge to create a frictionless transit booking experience. The primary motivations include: 1. **Accessibility:** Allowing travelers of all technical backgrounds to book intercity bus tickets via natural dialogue. 2. **Scientific Decision Support:** Replacing basic single-parameter sorting with a multi-criteria scoring engine that evaluates bus options across time, price, duration, availability, and quality ratings. 3.

Data Integrity: Enforcing strict atomicity in seat selection and human-in-the-loop confirmation gates to eliminate accidental or inconsistent bookings. 4. **Architectural Modularity:** Constructing a decoupled service layer that separates UI presentation from domain business logic.

2.4 Project Objectives

- **Deliver a Hybrid Dual-Mode Architecture:** Provide seamless parity between conversational AI booking and visual manual booking.
- **Build an Autonomous Travel Concierge ("Tixie"):** Implement a multi-turn Finite State Machine (FSM) supporting entity extraction, slot filling, contextual memory, and inline modifications.
- **Implement a Deterministic Recommendation Engine:** Create a 5-factor weighted scoring algorithm ($S_{\text{total}} = \sum w_i S_i$) for objective schedule ranking.
- **Ensure Atomic Seat Selection:** Enforce all-or-nothing seat locking to eliminate partial reservation states.
- **Incorporate Safety Confirmation Gates:** Require explicit, unambiguous user consent before any booking is staged or committed.
- **Simulate Real-Time Fleet Telemetry:** Render live bus movements on OpenStreetMap using waypoint interpolation and calculate device-to-bus distance using the Haversine formula.
- **Demonstrate Applied AI Engineering:** Showcase practical AI engineering, expert systems, and state-machine orchestration without external cloud dependencies.

3. Problem Statement

Modern online bus ticketing platforms suffer from core structural challenges:

1. **High Form Friction & Cognitive Load:** Booking a ticket requires navigating 5 to 7 sequential web forms (Origin $\rightarrow$ Destination $\rightarrow$ Date $\rightarrow$ Operator $\rightarrow$ Seat Map $\rightarrow$ Boarding/Dropping $\rightarrow$ Passenger Info $\rightarrow$ Checkout).
2. **Suboptimal Sorting & Recommendation:** Users are typically offered simplistic sort options (e.g., sort by price ascending), which often highlights slow, poorly rated buses with inconvenient departure times.
3. **Lack of Conversational Guidance:** Conventional chatbots on booking portals are restricted to static FAQ lookups and cannot execute multi-turn transactional workflows.
4. **Race Conditions & Partial Seat Reservation:** Concurrent seat selection in multi-passenger bookings can result in partial bookings if one seat becomes unavailable mid-flow.
5. **Disconnected Telemetry:** Travelers must rely on manual phone calls to drivers or third-party SMS alerts without integrated visual route tracking.

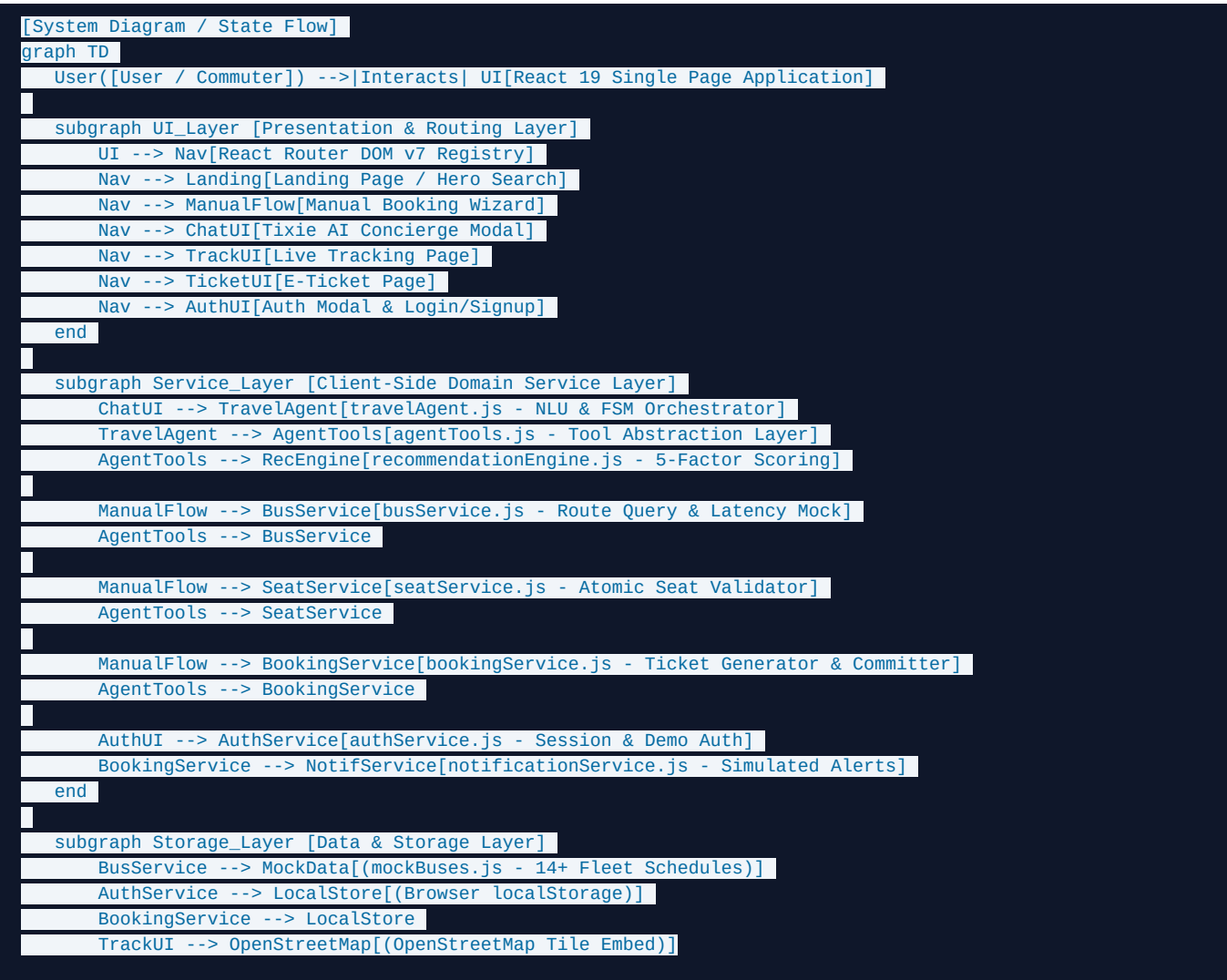
4. Existing Systems vs. Proposed GoTicket Platform

Dimension	Conventional Existing Portals	Proposed GoTicket Platform
Interaction Paradigm	Static web forms only	Dual Mode: Visual Manual Flow + Conversational AI Agent ("Tixie")
Search Method	Strict form dropdowns	Natural Language Understanding (NLU): Extracts cities, relative dates, time windows

Recommendation Engine	Single-column sorting (Price/Time)	5-Factor Weighted Scoring: Evaluates Time (30%), Price (25%), Duration (20%), Availability (15%), Rating (10%)
Seat Map Interaction	Visual grid only	Visual 40-seat interactive grid + Atomic conversational seat locking
Confirmation Safety	Direct progression to payment	Strict Explicit Confirmation Gate blocking accidental commits
Fleet Telemetry	SMS alerts / external links	Simulated Live OpenStreetMap Tracking with waypoint progression & Haversine user distance
User Verification	Complex authentication walls	Client session management + Simulated 6-digit mobile OTP verification
Software Architecture	Server-rendered monoliths	Modern React 19 SPA with decoupled domain service layer

5. System Architecture

GoTicket is structured as a layered, modular Single Page Application (SPA). Presentation components interface with domain business logic exclusively through a dedicated **Service Layer**, ensuring high cohesion and low coupling.



```
TrackUI --> GeolocationAPI[(HTML5 Geolocation API)]
```

```
end
```

6. Technology Stack

Layer	Technology	Version	Purpose in GoTicket	Evidence in Codebase
Frontend Framework	React	^19.1.1	Declarative component hierarchy and virtual DOM	package.json , src/App.js
DOM Renderer	React DOM	^19.1.1	Mounting and reconciliation in the browser DOM	package.json , src/index.js
Client Routing	React Router DOM	^7.8.2	Single-page client routing and state passing	package.json , src/App.js
Build Tooling	Create React App (react-scripts)	5.0.1	Webpack bundler, Babel transpiler, dev server	package.json
Programming Language	JavaScript (ES6+ / JSX)	ES2022	Application logic, async services, and NLU regex	src/**/*.js , src/**/*.jsx
Styling Architecture	CSS Modules & CSS Variables	Native CSS3	Scoped component styling and design tokens	src/styles/*.module.css , src/stylespages/*.module.css
NLU & AI Agent	Custom Regex NLU & FSM	Custom JS	Entity extraction, slot filling, state orchestration	src/services/travelAgent.js
Recommendation Engine	Multi-Criteria Scoring Algorithm	Custom JS	Deterministic 5-factor weighted bus ranking	src/services/recommendationEngi
Data Persistence	Web Storage API (localStorage)	Browser Native	Client session, registered users, coupons, tickets	src/services/bookingService.js , src/services/authService.js
Map Rendering	OpenStreetMap (Embed Iframe)	Leaflet / Mapnik	Visual route mapping and waypoint coordinates	src/pages/aboutgoticket/LiveTra
Geolocation	HTML5 Geolocation API	Browser Native	Device location lookup for Haversine distance	src/pages/aboutgoticket/LiveTra
Deployment Platform	Vercel Serverless Hosting	Cloud Platform	Production SPA hosting with route rewrite rules	vercel.json , .env
Version Control	Git & GitHub	Git 2.x	Distributed source control and collaboration	.git/config , README.md

7. Complete Project Directory Structure & Codebase Inventory

```
Go-Ticket/
├── .env # Environment configuration (CI=false)
├── .gitignore # Git ignore definitions
├── package.json # Project manifest, dependencies, scripts
├── package-lock.json # Lockfile for reproducible builds
├── README.md # Public repository documentation
├── vercel.json # Vercel SPA build output and route rewrite rules
├──
├── public/ # Static web assets
│   ├── favicon.ico # Browser tab icon
│   ├── index.html # Master HTML shell
│   ├── manifest.json # Web app manifest metadata
│   ├── robots.txt # Search crawler configuration
│   └── images/ # UI banners, bus vehicle graphics, logos
├──
├── docs/ # Comprehensive Academic Project Documentation
│   ├── PROJECT_REPORT.md # Master Final-Year Project Report
│   ├── TECHNICAL_DOCUMENTATION.md # Developer Architecture & Service Guide
│   ├── ARCHITECTURE.md # Mermaid Diagrams & Architecture Models
│   ├── VIVA_QUESTION.md # Comprehensive Viva Q&A Guide
│   ├── DEMO_SCRIPT.md # Step-by-step Project Presentation Script
│   └── TEAM_CONTRIBUTION.md # 4-Member Workload & File Ownership Matrix
├──
├── src/ # Application source code
│   ├── App.css # Global application stylesheet
│   ├── App.js # Master route registry and context provider wrapper
│   ├── App.test.js # Smoke testing suite
│   ├── index.css # Global typography, color tokens, and reset rules
│   ├── index.js # React application entry point (StrictMode mount)
│   ├── reportWebVitals.js # Web performance monitoring
│   └── setupTests.js # Jest testing environment setup
├──
├── context/ # Global React State Contexts
│   └── AuthContext.jsx # Authentication state provider (login, signup, logout)
├──
├── data/ # Static Mock Datasets
│   └── mockBuses.js # Master catalog of 14+ intercity buses with amenities
├──
├── services/ # Business Logic & Service Abstraction Layer
│   ├── agentTools.js # Tool interface connecting Tixie to domain logic
│   ├── authService.js # Session persistence, credentials verification
│   ├── bookingService.js # Ticket generation, pending handoff, localStorage commit
│   ├── busService.js # Bus query filter with simulated 500ms network latency
│   ├── notificationService.js # Simulated SMS & email notification logger
│   ├── recommendationEngine.js # Deterministic 5-factor weighted bus scoring engine
│   ├── seatService.js # 40-seat grid generator & atomic seat validator
│   └── travelAgent.js # Tixie multi-turn NLU, FSM, and confirmation gate
├──
├── components/ # Reusable UI Section Components
│   ├── About.jsx # Platform advantages & animated statistics counters
│   ├── AuthModal.jsx # Modal dialog hosting tabbed Login and Signup forms
│   ├── BookingSection.jsx # 4-step visual booking guide
│   ├── FAQ.jsx # Accordion frequently asked questions
│   ├── Features.jsx # Feature highlights grid
│   ├── Footer.jsx # Global navigation footer with contact links
│   ├── Header.jsx # Main header with trust badge, routes, auth avatar
│   ├── Hero.jsx # Search banner with 50+ city auto-suggestions
│   ├── Info.jsx # Informational view placeholder
│   ├── Layout.jsx # Layout wrapper rendering Header, Outlet, Footer, Chatbot
│   ├── OffersSection.jsx # Interactive promotional coupon gift modal
│   ├── TestimonialsSection.jsx # Marquee customer review carousel
│   └── TrackBus.jsx # Homepage live tracking promotional callout
├──
└── auth/ # Authentication Components
```


ProtectedRoute.jsx	# Guard wrapper prompting AuthModal if unauthenticated
Chatbot/	# Tixie Conversational AI Concierge UI
Chatbot.css	# Chatbot styles (launcher, bubble, chips, cards)
ChatbotLauncher.jsx	# Floating avatar button and notification tooltip
ChatModal.jsx	# Multi-turn chat interface, action cards, chips
assets/	
chatbot-girl.png	# Tixie character avatar graphic
pages/	# Routed Page Views
aboutgoticket/	# Core Journey & Information Pages
AvailableBuses.jsx	# Bus query results, AM/PM filters, slot selection
ContactUs.jsx	# 24/7 helpline cards, query submission, PNR lookup
DropPage.jsx	# Boarding & Dropping point radio selector
ETicket.jsx	# E-Ticket display, mobile OTP lookup, PDF print
Home.jsx	# Popular intercity routes explorer & price comparison
LiveTracking.jsx	# OpenStreetMap live tracking, waypoints, Haversine dist
PaymentPage.jsx	# Multi-passenger checkout, OTP verify, promo discounts
SeatBooking.jsx	# Standalone route & date search page
SelectSeats.jsx	# 40-seat interactive bus grid with sold locks
auth/	# Authentication Pages
Login.jsx	# Dedicated Login page with demo credentials button
Signup.jsx	# Dedicated Signup registration page
Infogo-ticket/	# Legal & Policy Information Pages
NonDisclosureAggrement.jsx	# NDA policy view
PrivacyPolicy.jsx	# Privacy policy view
Refernce.jsx	# Academic & transit references view
ResponsibleClosure.jsx	# Responsible disclosure policy view
Terms&Conditions.jsx	# Terms and conditions view
WorkingCriteria.jsx	# Operational criteria view
styles/	# Component-Scoped CSS Modules
stylespages/	# Page-Scoped CSS Modules

8. User Roles & Access Architecture

In the current implementation, GoTicket supports three operational user roles:

- Guest User (Unauthenticated):** * Can perform route searches, browse schedules, view bus amenities, interact with Tixie for queries, and explore live tracking simulation. * Accessing protected workflows prompts the `AuthModal` dialog or redirects to login.
- Authenticated Demo User:** * Can complete end-to-end seat bookings, apply discount vouchers, generate digital E-Tickets, and retrieve historical bookings using registered credentials. * Built-in pre-configured demo account: `demo@goticket.in / demo123`.
- Passenger / Traveler Entity:** * Represents individual passenger records associated with a ticket object (Full Name, Gender, Age, Mobile, Masked Aadhaar). Multiple passengers can be registered under a single ticket.

9. Authentication & Session Management

9.1 Overview & State Flow

Authentication is managed globally via `AuthContext.jsx` and persisted via `authService.js`. The state exposes `{ user, isAuthenticated, loading, error, login, signup, logout }`.

[System Diagram / State Flow]
sequenceDiagram

```
    autonumber
    actor User
    participant UI as Login / Signup Component
    participant Context as AuthContext.jsx
    participant Service as authService.js
    participant Storage as localStorage
    [
    User->>UI: Submits Email & Password
    UI->>Context: login(email, password)
    Context->>Service: authService.login(email, password)
    [
    alt Demo Credentials (demo@goticket.in / demo123)
        Service->>Storage: _persistSession(user)
        Service-->>Context: Returns Demo User Object
    else Registered in localStorage['registeredUsers']
        Service->>Storage: Find user in registeredUsers
        Service->>Storage: _persistSession(user)
        Service-->>Context: Returns Registered User Object
    else Invalid Credentials
        Service-->>Context: Throws Error ("Invalid email or password")
        Context-->>UI: Sets error state
    end
    [
    Context-->>UI: Updates isAuthenticated = true
    UI-->>User: Closes Modal / Navigates to Target Route
```

9.2 Registration (Signup Flow)

- **Fields:** Full Name, Email, 10-Digit Mobile, Password, Confirm Password.
- **Validation:**
 - Full Name: Minimum 2 characters.
 - Email: Valid RFC 5322 regex.
 - Mobile: 10-digit Indian standard starting with 6–9 (`^[6-9]\d{9}<code>`).
 - Password: Minimum 6 characters with confirmation check.
- **Persistence:** Appended to `localStorage.getItem('registeredUsers')`.

9.3 Security & Academic Disclosure

- Passwords are stored in plain text inside browser `localStorage` for prototype demonstration.
- Session tokens are client-generated mock strings (`'token-' + Date.now()`).
- In production, password hashing (bcrypt) and asymmetric JWTs must be managed by a secure backend server.

10. Manual Booking Workflow (End-to-End)

```
[System Diagram / State Flow]
flowchart TD
    A[Hero.jsx / SeatBooking.jsx] -->|Select Source, Destination, Date| B[AvailableBuses.jsx]
    B -->|Filter AM/PM, Select Time Slot| C[SelectSeats.jsx]
    C -->|Choose S1..S40, Validate Sold Seats| D[DropPage.jsx]
    D -->|Select Boarding & Dropping Points| E[PaymentPage.jsx]
    E -->|Add Passengers, Verify 6-digit OTP| F[Payment & Promo Code]
    F -->|Apply FIRSTGO / GTWEEKEND, Pay via UPI / Card| G[Confirmation & ETicket.jsx]
```

Step-by-Step Breakdown

1. **Search Initiation (Hero.jsx / SeatBooking.jsx):** * User inputs origin and destination cities from 50+ auto-suggestions. * Validates non-empty input, distinct cities, and non-past dates. * Passes state { from, to, date, ampm } to /available-buses.
2. **Schedule Selection (AvailableBuses.jsx):** * Calls busService.searchBuses(). Displays operators, ratings, amenities, and departure slots. * User selects a slot (e.g., 08:30 PM (■680)) and navigates to /select-seats.
3. **Interactive Seat Selection (SelectSeats.jsx):** * Renders 40 seats in an 8-row \times 5-column layout (2 Left + Aisle + 2 Right). * Checks seatService.DEFAULT_SOLD_SEATS (['S2', 'S7', 'S12', 'S18', 'S24', 'S31']) and locks them in disabled gray. * Dynamic fare updates: \text{Total} = \text{Seat Count} \times \text{Slot Fare}.
4. **Boarding & Dropping Selector (DropPage.jsx):** * User chooses route-specific pickup and drop locations via radio buttons.
5. **Passenger Details & OTP (PaymentPage.jsx – Step 1 & 2):** * Collects Full Name, Gender, Age, Mobile, Email, and 12-digit Aadhaar. * Clicking "Send OTP" generates a 6-digit demo code displayed via browser alert(). * Entering the code marks mobileVerified = true.
6. **Checkout & Coupons (PaymentPage.jsx – Step 3):** * Promotional coupons: FIRSTGO (■150 OFF), GTWEEKEND (■200 OFF), UPIPAY (■50 OFF). * Payment method: UPI ID (validated with @) or Card (16 digits, CVV, expiry).
7. **Finalization & Ticket Generation (PaymentPage.jsx – Step 4 & ETicket.jsx):** * Calls finalizeBooking() in bookingService.js. * Generates ticket reference GTXXXXXX and commits to localStorage['lastTicket'].

11. Bus Recommendation Engine

The recommendation engine (src/services/recommendationEngine.js) implements a deterministic multi-criteria scoring algorithm to rank bus schedules scientifically.

11.1 Multi-Criteria Scoring Formula

$$\text{Total Score} = (S_{\text{time}} \times 0.30) + (S_{\text{price}} \times 0.25) + (S_{\text{duration}} \times 0.20) + (S_{\text{seats}} \times 0.15) + (S_{\text{rating}} \times 0.10)$$

Where: * **Time Match Score (S_{time}), Weight = 30%:** $S_{\text{time}} = \max\left(0, 1 - \frac{|\text{Bus Dep. Minutes} - \text{Preferred Time Minutes}|}{360}\right)$ * **Price Score (S_{price}), Weight = 25%:** $S_{\text{price}} = 1 - \frac{\text{Bus Price} - \text{Min Price}}{\text{Max Price} - \text{Min Price}}$ * **Duration Score (S_{duration}), Weight = 20%:** $S_{\text{duration}} = \max\left(0.5, 1 - \frac{\text{Duration in Hours} - 2}{10}\right)$ * **Seat Availability Score (S_{seats}), Weight = 15%:** $S_{\text{seats}} = \min\left(1.0, \frac{\text{Available Seats}}{30}\right)$ * **Rating Score (S_{rating}), Weight = 10%:** $S_{\text{rating}} = \frac{\text{Customer Rating}}{5.0}$

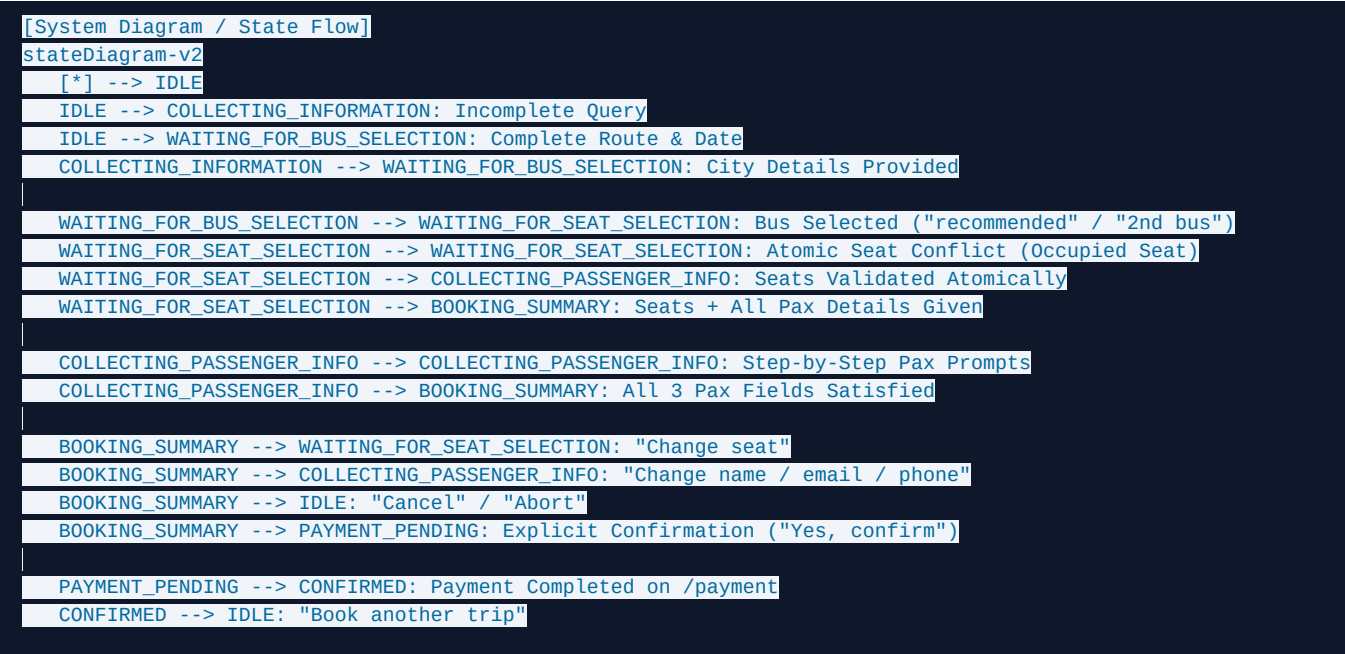
12. Tixie – Conversational AI Travel Agent

Tixie is the multi-turn conversational AI travel concierge implemented in src/services/travelAgent.js and rendered via src/components/Chatbot/ChatModal.jsx.

12.1 Natural Language Understanding (NLU) Pipeline

Tixie executes deterministic regex tokenization to extract entities: * **Cities:** Matches 22 Indian cities across patterns like from <city> to <city> and <city> -> <city>. * **Dates:** Resolves relative keywords (today, tomorrow, day after tomorrow, and days of the week). * **Time Windows:** Identifies specific times (9 PM, 08:30 AM) and periods (morning, evening, night). * **Seat IDs:** Regex \bS[-_s]?d{1,2}\b maps to S1-S40. * **Passenger Data:** Regex extraction for full name, email, and 10-digit Indian phone number.

12.2 Finite State Machine (FSM) Specification



12.3 FSM State Transition Table

State Name	Purpose	Entry Condition	Expected User Input	Action / Output	Next State
IDLE	Initial idle listening state	App launch or post-cancellation	Travel query (e.g., "Buses to Delhi")	Parses travel parameters	COLLECTING_INFORMATION or WAITING_FOR_BUS_SELECTION
COLLECTING_INFORMATION	Prompts for missing route fields	Missing source or destination	City name ("From Kanpur")	Updates travel parameters	WAITING_FOR_BUS_SELECTION
WAITING_FOR_BUS_SELECTION	Presents ranked bus options	Search results ready	Selection intent ("1st one", "cheapest")	Calls get_bus_details and checks seat map	WAITING_FOR_SEAT_SELECTION
WAITING_FOR_SEAT_SELECTION	Shows and validates seat IDs	Bus selected	Seat labels ("S3 and S4")	Calls hold_select_seats (Atomic validation)	COLLECTING_PASSENGER_INFO or BOOKING_SUMMARY
COLLECTING_PASSENGER_INFO	Prompts for step-by-step passenger data entry	Seats validated	Name, Email, or 10-digit Phone	Validates format per field	COLLECTING_PASSENGER_INFO or BOOKING_SUMMARY
BOOKING_SUMMARY	Transparent fare & pax summary	All details gathered	Explicit confirmation or modification	Presents action card and summary	PAYMENT_PENDING, WAITING_FOR_SEAT_SELECTION, or IDLE
PAYMENT_PENDING	Prepares booking for checkout	Explicit confirmation received	Clicks "Proceed to Payment"	Hands off booking object to /payment	CONFIRMED

BOOKING	In-flight duplicate lock	Request being committed	None (Transient lock)	Disables duplicate clicks	CONFIRMED
CONFIRMED	Finalized booking confirmation	Payment finalized	Post-booking query	Displays ticket ID and E-Ticket link	IDLE

12.4 Strict Explicit Confirmation Gate

Tixie enforces a strict safety protocol: * **Accepted Confirmation Phrases:** "confirm", "yes", "yes confirm", "yes, confirm", "yes please", "yes book it", "book it", "confirm booking", "go ahead", "proceed", "i want to book it". * **Non-Committal Input:** Phrases like "maybe" or "what is the fare?" do NOT commit a booking; Tixie preserves state and re-prompts for explicit approval. * **Inline Modifications:** Users can modify seats ("change seat to S5"), update emails ("change email"), or cancel ("abort") at any point.

13. Seat Management & Atomic Validation

Seat management logic (src/services/seatService.js) enforces the **Principle of Atomicity**:

- **Bus Layout:** 40 seats (S1 to S40).
- **Occupied Set:** DEFAULT_SOLD_SEATS = ['S2', 'S7', 'S12', 'S18', 'S24', 'S31'].
- **Atomic Rule:** If a user requests multiple seats (e.g., ["S5", "S7"]) and any seat is occupied (S7), the entire reservation is rejected atomically. No partial seats are held.

14. Live Bus Tracking & OpenStreetMap Integration

- **Catalog of Predefined Routes (BUS_FLEET):**
- UP32AB1234 (Lucknow \to Kanpur): Lucknow ISBT, Unnao, Kanpur Central.
- DL01CD5678 (Delhi \to Agra): Delhi ISBT, Mathura, Agra Fort.
- MH12EF9012 (Mumbai \to Pune): Mumbai Dadar, Lonavala, Pune Swargate.
- **Simulation Loop:** A 3000ms setInterval timer increments vehicle progress by +0.3% per tick, calculating coordinates via linear interpolation.
- **OpenStreetMap Integration:** Embeds an OSM export iframe bounded by the vehicle's dynamic bounding box.
- **User Distance Calculation:** Invokes navigator.geolocation.getCurrentPosition and calculates distance via the **Haversine Formula**: $d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta\text{lng}}{2}\right)}\right)$

Technical Disclosure: Fleet tracking is 100% simulated client-side using waypoint interpolation and is not connected to physical GPS hardware.

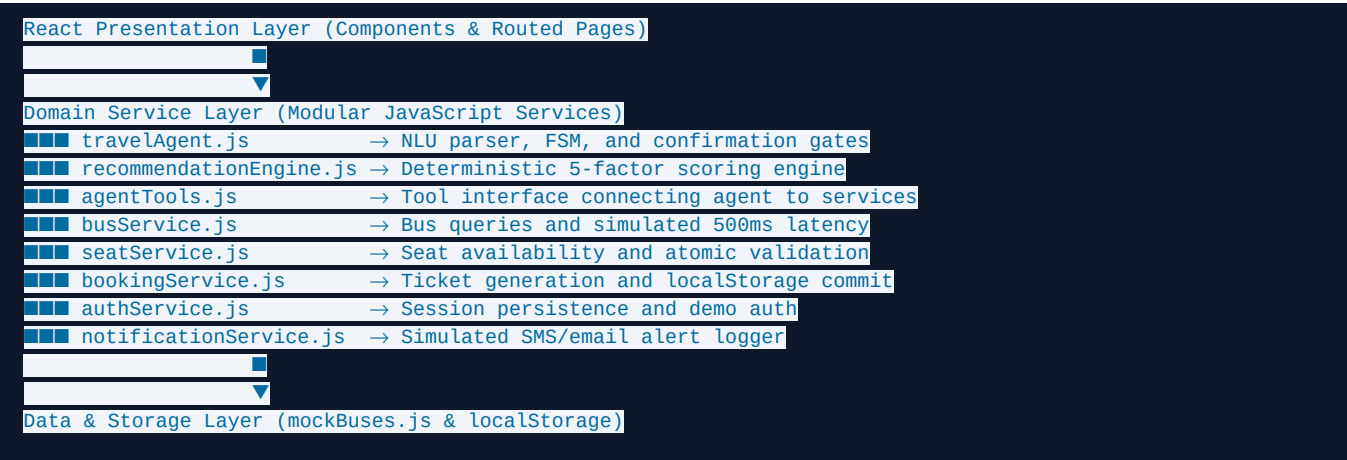
15. LocalStorage Schema & Persistence Architecture

Storage Key	Type	Schema / Stored Attributes	Managing File	Survives Reload?
-------------	------	----------------------------	---------------	------------------

lastTicket	JSON	{ ticketId, name, route, date, time, totalFare, seats, passenger, passengers, bookedAt }	bookingService.js	Yes
pendingBooking	JSON	{ busDetails, seats, totalFare, passenger, appliedCoupon, discountAmount }	bookingService.js	Yes
registeredUsers	JSON Array	[{ name, email, mobile, password, registeredAt }]	authService.js	Yes
authUser	JSON	{ name, email, mobile, token }	authService.js	Yes
appliedCoupon	JSON	{ code, discountText, discount }	OffersSection.jsx	Yes

16. Service Layer & Separation of Concerns

The architecture strictly decouples React UI views from underlying business logic:



17. Routing Registry

Route Path	Component	Description	Protected?
/	Layout.jsx (Hero, Offers, About, Booking, TrackBus, Testimonials, FAQ)	Main Landing & Discovery Page	No
/home	Home.jsx	Popular Intercity Bus Corridors Explorer	No

/booking	SeatBooking.jsx	Route Search & Travel Date Selection View	No
/seatbooking	SeatBooking.jsx	Standalone Route & Date Search Form	No
/available-buses	AvailableBuses.jsx	Bus Search Results with AM/PM Slot Filters	No
/select-seats	SelectSeats.jsx	40-Seat Interactive Bus Layout Map	No
/drop	DropPage.jsx	Boarding & Dropping Point Radio Selector	No
/payment	PaymentPage.jsx	Multi-Passenger Checkout, OTP, and Payment Flow	No (In-flow guards)
/eticket	ETicket.jsx	E-Ticket Display, Mobile Lookup, and PDF Print	No
/livetracking	LiveTracking.jsx	OpenStreetMap Bus Telemetry & Haversine Distance	No
/contact	ContactUs.jsx	24/7 Support Desk, PNR Lookup, and Feedback Form	No
/login	Login.jsx	User Sign In View with Demo Autofill	No
/signup	Signup.jsx	New User Registration Form	No
/privacy	PrivacyPolicy.jsx	Privacy Policy Statement	No
/terms	Terms&Conditions.jsx	Terms & Conditions Agreement	No
/nda	NonDisclosureAggrement.jsx	Non-Disclosure Agreement Statement	No
/reference	Refernce.jsx	Academic & Technical References	No
/responsible-closure	ResponsibleClosure.jsx	Responsible Disclosure Policy	No
/working-criteria	WorkingCriteria.jsx	Platform Operational Criteria	No

18. Comprehensive Test Suite & Test Execution Matrix

Test ID	Module	Test Scenario	Input Data	Expected Result	Actual Result	Status
TC-AUTH-01	Auth	Valid Demo Login	demo@goticket.in / demo123	Authenticates successfully, stores session	User session stored, avatar updated	PASSED
TC-AUTH-02	Auth	Invalid Password Login	demo@goticket.in / wrongpass	Throws error banner	Displays "Invalid email or password"	PASSED
TC-AUTH-03	Auth	New User Registration	Valid name, email, 10-digit mobile, 6+ pass	Creates user in registeredUsers	Account created, auto logged-in	PASSED
TC-AUTH-04	Auth	Mismatched Signup Passwords	Password 123456, Confirm 654321	Validation error banner	Displays "Passwords do not match"	PASSED
TC-SEARCH-01	Bus Search	Valid Search Query	Source: Kanpur, Dest: Delhi, Date: Tomorrow	Returns matching buses from MOCK_BUSES	3 buses returned with schedules	PASSED
TC-SEARCH-02	Bus Search	Identical Cities Search	Source: Kanpur, Dest: Kanpur	Rejects search with warning	"Departure and destination cannot be same"	PASSED
TC-SEAT-01	Seat Selection	Available Seat Selection	Seat S3, S4	Seats marked selected, fare calculated	Seats highlighted green, fare updated	PASSED
TC-SEAT-02	Seat Selection	Occupied Seat Selection	Seat S7 (Sold in DEFAULT_SOLD_SEATS)	Click disabled, Seat stays gray	Seat cannot be clicked	PASSED
TC-SEAT-03	Seat Service	Atomic Multi-Seat Validation	Request ['S5', 'S7'] (S7 occupied)	Whole request rejected atomically	Fails atomically; S5 not held	PASSED

TC-TIXIE-01	AI Agent	Natural Language Route Query	"Find buses from Kanpur to Delhi tomorrow"	Extracts route & date, returns ranked buses	Displays top buses + recommendation reasons	PASSED
TC-TIXIE-02	AI Agent	Incomplete Query Handling	"Find buses to Delhi"	Prompts for missing departure city	Asks: "Where will you be traveling from?"	PASSED
TC-TIXIE-03	AI Agent	Sequential Pax Collection	Step 1: Name, Step 2: Email, Step 3: Phone	Prompts for each missing field in sequence	Collects all 3 before generating summary	PASSED
TC-TIXIE-04	AI Agent	Single-Sentence Pax Input	"Name is Anshika, anshika@test.com, 9876543210"	Extracts all 3 fields in single turn	Generates Booking Summary immediately	PASSED
TC-TIXIE-05	AI Agent	Strict Confirmation Gate	Ambiguous query "Maybe" in summary state	Does NOT create booking; re-prompts	Warns and prompts for explicit confirmation	PASSED
TC-TIXIE-06	AI Agent	Explicit Confirmation Approval	User says "Yes, confirm"	Prepares booking, hands off to /payment	Transitions to <code>PAYMENT_PENDING</code>	PASSED
TC-TIXIE-07	AI Agent	Inline Detail Modification	"Change seat to S5" in summary state	Returns to seat state without resetting route	Seats updated to S5	PASSED
TC-PAY-01	Payment	Mobile OTP Verification	Valid 10-digit mobile, correct OTP code	Marks <code>mobileVerified = true</code>	Verified badge shown, unlocks payment step	PASSED
TC-PAY-02	Payment	Promotional Coupon Apply	Applies coupon code <code>FIRSTG0</code>	Deducts <code>₹150</code> from total payable	Total fare reduced by <code>₹150</code>	PASSED
TC-PAY-03	Payment	Successful Checkout	UPI user@upi, completes pay	Finalizes ticket, generates <code>GTXXXXXX</code>	Persists to <code>lastTicket</code> , opens E-Ticket	PASSED

TC-TRACK-01	Tracking	Simulated Waypoint Move	Bus UP32AB1234	3s timer advances progress along route	Position marker moves, distance updates	PASSED
TC-TRACK-02	Tracking	Geolocation Haversine	User grants browser location permission	Computes distance in km to bus	Displays real-time km distance	PASSED

19. Limitations & Future Scope

19.1 Current Limitations

- **No Persistent Backend Server:** All business logic and mock API latency run in client JavaScript.
- **Client-Only Persistence:** Data is stored in browser localStorage and does not sync across multiple devices.
- **Simulated External Integrations:** Payment gateway (Razorpay), SMS delivery (Twilio), and live GPS feeds are simulated for prototype evaluation.
- **Deterministic NLU:** Tixie uses regex rules and FSM transitions rather than an external cloud LLM.

19.2 Future Roadmap

- **Backend Layer:** Build a high-performance **FastAPI (Python)** or **Express (Node.js)** REST API.
- **Database Engine:** Integrate **PostgreSQL** or **MySQL** with SQLAlchemy ORM.
- **Authentication:** Implement cryptographically signed **JWTs** with bcrypt password hashing.
- **External LLM:** Connect Tixie to **Google Gemini API** with structured Function Calling.
- **Live GPS Feeds:** Stream real-time vehicle telematics via **WebSockets / MQTT**.

20. AI / ML Relevance & Engineering Concepts

GoTicket demonstrates key concepts in Artificial Intelligence & Software Engineering: 1. **Multi-Criteria Decision Analysis (MCDA):** Deterministic mathematical weighting ($S_{\text{total}} = \sum w_i S_i$) for objective schedule recommendation. 2. **Goal-Oriented Finite State Automata:** Dialogue State Tracking (DST) and slot filling without LLM hallucinations. 3. **Atomic State Transactions:** Guaranteed atomicity in seat selection to prevent data fragmentation. 4. **Human-in-the-Loop AI Safety:** Strict confirmation gate preventing unauthorized financial transactions.

21. Team Contribution Matrix

Member	Primary Scope	Assigned Source Files
Member 1 (Anshika Pandey)	AI Travel Agent & NLU	src/services/travelAgent.js src/services/recommendationEngine.js src/services/agentTools.js src/components/Chatbot/*

Member 2 (Jaishree Verma)	Frontend & UI/UX	src/components/Hero.jsx src/components/Header.jsx src/components/Footer.jsx src/pages/aboutgoticket/SelectSeats.jsx src/styles/*, src/stylespages/*
Member 3 ([Student 3])	Booking Engine & Checkout	src/pages/aboutgoticket/AvailableBuses.jsx src/pages/aboutgoticket/DropPage.jsx src/pages/aboutgoticket/PaymentPage.jsx src/pages/aboutgoticket/ETicket.jsx src/services/bookingService.js
Member 4 ([Student 4])	Telemetry, Auth & Deploy	src/pages/aboutgoticket/LiveTracking.jsx src/context/AuthContext.jsx src/services/authService.js src/services/notificationService.js vercel.json

22. Conclusion

GoTicket successfully delivers an end-to-end, dual-mode intercity bus booking and fleet tracking platform that unifies traditional web UI efficiency with modern conversational AI concierge capabilities. Through its deterministic 5-factor recommendation engine, atomic seat reservation safeguards, strict confirmation gates, simulated live telemetry, and clean service-oriented architecture, GoTicket demonstrates robust software engineering and applied AI principles suitable for a final-year B.Tech project.

PART 2: TECHNICAL & DEVELOPER DOCUMENTATION

GoTicket — Technical & Developer Documentation

1. System Overview & Developer Architecture

GoTicket is structured as a client-side Single Page Application (SPA) built with **React 19** and **React Router DOM v7**. The application decouples presentation components from domain logic using an explicit **Domain Service Layer** located in `src/services/`.

1.1 Core Architectural Principles

- Separation of Concerns:** React components handle DOM rendering, user event dispatching, and view-local UI state. All business operations (bus querying, deterministic ranking, atomic seat validation, booking preparation, session storage, and notification logging) are encapsulated within modular JavaScript services.
- Deterministic Processing:** The conversational agent ("Tixie") and recommendation engine operate deterministically via regular expressions, finite state machines, and mathematical scoring functions, avoiding unpredictable LLM hallucinations or external API costs.
- Data Integrity via Atomicity:** Seat holding logic enforces strict all-or-nothing validation (`validateAndHoldSeats`).
- Backend-Ready Interface Contracts:** Service functions return standard `Promise`-based structures (`{ success, data, error }`), enabling seamless transition to a live backend (FastAPI / Express) by swapping internal mock queries with Axios / Fetch API calls.

2. Comprehensive Service Layer Documentation

2.1 `travelAgent.js` — Conversational NLU & State Orchestrator

- File Location:** `src/services/travelAgent.js`
- Total Lines:** ~1,130 lines
- Responsibility:** Serves as the master conversational engine ("Tixie"). Parses natural language queries, manages dialogue state transitions, executes tool calls, enforces the explicit confirmation safety gate, and prepares booking contexts for payment handoff.

Exported Functions

- `extractSeatIDs(text: string): string[]` * Extracts normalized seat labels (e.g., "S3", "S4", "seats 3 and 4") using regex `/\bS[-_\\s]?\\d{1,2}\\b/gi` and `/\\bseats?\\s*(\\d{1,2}\\s*)/gi`. * Returns deduplicated uppercase strings (`['S3', 'S4']`).
- `extractTravelParams(text: string): TravelParamsObject` * Extracts `source`, `destination`, `date`, `preferredTime`, and `isArrivalConstraint`. * Matches 22 Indian cities (`CITIES` array), relative dates (`today`, `tomorrow`, `day after tomorrow`, days of the week), and time windows (`morning`, `evening`, `08:30 PM`).
- `processAgentMessage(userMessage: string, agentState: Object): Promise<AgentResponseObject>` * Main asynchronous entry point for processing conversational turns. * Manages dialogue states: `IDLE`, `COLLECTING_INFORMATION`, `WAITING_FOR_BUS_SELECTION`, `WAITING_FOR_SEAT_SELECTION`, `COLLECTING_PASSENGER_INFO`, `BOOKING_SUMMARY`, `PAYMENT_PENDING`, `BOOKING`, `CONFIRMED`. * Returns `{ agentState, statusTrace, text, actionCard, chips, searchResults, recommendation }`.

2.2 recommendationEngine.js — Multi-Criteria Scoring Algorithm

- **File Location:** `src/services/recommendationEngine.js`
- **Responsibility:** Implements a deterministic Multi-Criteria Decision Analysis (MCDA) scoring algorithm that evaluates and ranks bus search results based on user preferences.

Scoring Formula & Normalization

$$\text{Total Score} = (S_{\text{time}} \times 0.30) + (S_{\text{price}} \times 0.25) + (S_{\text{duration}} \times 0.20) + (S_{\text{seats}} \times 0.15) + (S_{\text{rating}} \times 0.10)$$

- **Time Match Score (S_{time} – 30%):** Computed by converting departure time to minutes from midnight and penalizing deviation from preferred time over a 360-minute window.
- **Price Score (S_{price} – 25%):** Normalized linearly against the minimum and maximum fares in the current search results.
- **Duration Score (S_{duration} – 20%):** Evaluates journey duration relative to a baseline curve.
- **Seat Availability Score (S_{seats} – 15%):** Proportional to available seats normalized up to 30 open seats.
- **Rating Score (S_{rating})\$ – 10%):** Customer rating divided by 5.0.

Exported Functions

- `rankBuses(buses: Array, preferences: Object): { bestBus, score, rankedBuses, reasons }`

2.3 agentTools.js — Tool Abstraction Layer

- **File Location:** `src/services/agentTools.js`
- **Responsibility:** Provides standardized tool wrappers for Tixie, interfacing with domain services.

Exported Tools

1. `search_buses({ source, destination, date, preferredTime }): Promise<ToolResult>` * Validates parameters, executes `busService.searchBuses()`, and returns structured results.
2. `get_bus_details({ busId }): Promise<ToolResult>` * Looks up complete bus metadata by ID or name from `mockBuses.js`.
3. `check_seat_availability({ busId, date, slot }): Promise<ToolResult>` * Fetches available and occupied seats via `seatService.getAvailableSeats()`.
4. `hold_select_seats({ busId, date, slot, seats, farePerSeat }): Promise<ToolResult>` * Atomically validates requested seats against `seatService.validateAndHoldSeats()`.
5. `prepare_booking(params): Promise<ToolResult>` * Validates final seat availability and creates pending booking context in `localStorage['pendingBooking']` for payment handoff.
6. `create_booking(params): Promise<ToolResult>` * Executes final atomic seat validation and commits booking directly via `bookingService.createAgentBooking()`.

2.4 bookingService.js — Booking Lifecycle & Ticket Schemas

- **File Location:** `src/services/bookingService.js`
- **Responsibility:** Manages pending booking staging, final ticket generation, and Web Storage persistence.

Exported Functions

1. `generateTicketId(): string` * Generates reference ID: `'GT' + Math.random().toString(36).substring(2, 8).toUpperCase()` (e.g., GT7AX9KP).
2. `preparePendingBooking(opts): { success, pendingBooking, error }` * Stages booking data and writes to `localStorage['pendingBooking']`.
3. `finalizeBooking({ bookingData, paymentMethod, passengers, ticketId }): { success, ticketId, ticket }` * Commits finalized ticket to `localStorage['lastTicket']` and purges `pendingBooking`.
4. `createAgentBooking(opts): Object` * Convenience wrapper combining preparation and direct commit.
5. `getPendingBooking(): Object|null` * Reads and parses `localStorage['pendingBooking']`.
6. `clearPendingBooking(): void` * Removes `pendingBooking` key.
7. `getLastTicket(): Object|null` * Reads and parses `localStorage['lastTicket']`.

2.5 `seatService.js` — Seat Grid & Atomic Validator

- **File Location:** `src/services/seatService.js`
- **Responsibility:** Single source of truth for the 40-seat bus layout and atomic validation.

Constants & Exported Functions

- `TOTAL_SEATS_COUNT = 40`
- `DEFAULT_SOLD_SEATS = ['S2', 'S7', 'S12', 'S18', 'S24', 'S31']`
- `getAllSeats(): string[]` — Returns `['S1', 'S2', ..., 'S40']`.
- `getAvailableSeats(busId, date, slot): { busId, date, slot, totalSeats: 40, availableSeats, occupiedSeats }`
- `validateAndHoldSeats(busId, date, slot, requestedSeats, farePerSeat): Object`
- **Atomicity Rule:** If any requested seat is occupied or invalid, `atomicityHeld = false`, `success = false`, and the entire request is rejected with descriptive error text.

2.6 `busService.js` — Route Search & Query Filter

- **File Location:** `src/services/busService.js`
- **Responsibility:** Filters `MOCK_BUSES` by source, destination, date, max price, and bus type. Simulates 500ms network latency via `setTimeout` to test loading states.

Exported Functions

- `searchBuses(searchParams): Promise<Array<BusObject>>`

2.7 `authService.js` — Authentication & Session Service

- **File Location:** `src/services/authService.js`
- **Responsibility:** Validates credentials against demo data (`demo@goticket.in / demo123`) and `localStorage['registeredUsers']`. Persists active session under `localStorage['authUser']`.

Exported Methods

- `login(email, password): Promise<UserObject>`
- `signup({ name, email, mobile, password }): Promise<UserObject>`
- `getCurrentUser(): UserObject|null`
- `logout(): Promise<void>`

2.8 notificationService.js — Simulated Alert Dispatcher

- **File Location:** `src/services/notificationService.js`
- **Responsibility:** Formats and logs simulated booking confirmation emails and SMS messages.

Exported Functions

- `maskEmail(email: string): string` — e.g., "an***@gmail.com".
- `maskMobile(mobile: string): string` — e.g., "*****3210".
- `sendTicketEmail({ email, ticket }): Promise<{ attempted, success, mode: 'demo', preview }>`
- `sendTicketSMS({ mobile, ticket }): Promise<{ attempted, success, mode: 'demo', preview }>`

3. Data Models & JSON Schemas

3.1 Finalized Ticket Schema (`localStorage['lastTicket']`)

```
{
  "ticketId": "GT88A19B",
  "name": "GoRide Express",
  "label": "GoRide Express",
  "id": "UP78KN1234",
  "type": "AC Sleeper 2+1",
  "route": "Kanpur => Delhi",
  "date": "2026-09-12",
  "time": "08:30 PM",
  "fare": 680,
  "seats": ["S3", "S4"],
  "boarding": {
    "time": "04:30 PM",
    "location": "Jhatkari Bus Station",
    "address": "GT Road, Kanpur",
    "popular": true
  },
  "dropping": {
    "time": "06:15 PM",
    "location": "Transport Nagar Metro Station",
    "address": "Near Gate No. 1, Lucknow",
    "popular": true
  },
  "totalFare": 1210,
  "appliedCoupon": "FIRSTGO",
  "discountAmount": 150,
  "paymentMethod": "upi",
  "passenger": {
    "fullName": "Anshika Pandey",
    "mobile": "9876543210",
    "email": "anshika@example.com",
    "aadhaar": "*****9012",
    "gender": "Female",
```



```
    "age": 22
  },
  "passengers": [
    {
      "fullName": "Anshika Pandey",
      "mobile": "9876543210",
      "email": "anshika@example.com",
      "aadhaar": "*****9012",
      "gender": "Female",
      "age": 22
    }
  ],
  "bookedAt": "2026-09-11T18:00:00.000Z"
}
```

3.2 Bus Object Schema (src/data/mockBuses.js)

```
{
  "id": "UP78KN1234",
  "operator": "GoRide Travels",
  "busName": "GoRide Express",
  "source": "Kanpur",
  "destination": "Delhi",
  "departureTime": "08:30 PM",
  "arrivalTime": "05:30 AM",
  "duration": "9h 00m",
  "price": 680,
  "availableSeats": 18,
  "busType": "AC Sleeper 2+1",
  "rating": "4.8 ★",
  "amenities": ["■ Charging", "■ Free Wi-Fi", "■ Water Bottle", "■ Reclining Seats"],
  "slots": [
    { "time": "08:30 PM", "fare": "■680", "status": "available" },
    { "time": "09:30 PM", "fare": "■720", "status": "filling" },
    { "time": "10:30 PM", "fare": "■750", "status": "available" }
  ]
}
```

4. Frontend Component Hierarchy & React Lifecycle

```
App.jsx (AuthProvider > Router > Routes)
  ■■■ Layout.jsx
    ■■■ Header.jsx (Navigation, Auth Avatar, Login Modal Trigger)
    ■■■ Outlet (Dynamic Page Router)
      ■ ■■■ Home.jsx (Corridor Explorer)
      ■ ■■■ Hero.jsx + OffersSection + About + BookingSection + TrackBus + Testimonials + FAQ
      ■ ■■■ AvailableBuses.jsx (Schedule Listing & Slot Picker)
      ■ ■■■ SelectSeats.jsx (40-Seat Bus Layout Grid)
      ■ ■■■ DropPage.jsx (Boarding & Dropping Point Radio Selection)
      ■ ■■■ PaymentPage.jsx (Multi-Passenger Wizard & Simulated Checkout)
      ■ ■■■ ETicket.jsx (Printable Pass & QR View)
      ■ ■■■ LiveTracking.jsx (OpenStreetMap Telemetry)
      ■ ■■■ ContactUs.jsx (Support Hotline & PNR Status Check)
      ■ ■■■ Login.jsx / Signup.jsx (Auth Views)
      ■ ■■■ Policy Pages (Privacy, Terms, NDA, Reference, Closure, Criteria)
      ■■■ Footer.jsx (Global Links & Helpline)
      ■■■ ChatbotLauncher.jsx (Floating Avatar Bubble)
        ■■■ ChatModal.jsx (Tixie Conversational UI & Action Cards)
```

5. Transitioning to a Live Production Backend

To convert GoTicket into a full-stack production application: 1. **Backend Server:** Develop a FastAPI or Express server implementing endpoints: * `POST /api/v1/auth/login` and `POST /api/v1/auth/signup` (with bcrypt and JWT). * `GET /api/v1/buses/search?source=&destination=&date=.` * `GET /api/v1/seats/:busId?date=&slot=.` * `POST /api/v1/bookings/prepare` and `POST /api/v1/bookings/finalize`. 2. **Service Layer Updates:** Replace internal `localStorage` / `setTimeout` queries in `authService.js`, `busService.js`, `seatService.js`, and `bookingService.js` with Axios calls pointing to `process.env.REACT_APP_API_URL`. 3. **UI Unaffected:** Zero modifications will be required in React UI components, demonstrating the architectural strength of the decoupled service layer.

PART 3: SYSTEM ARCHITECTURE & DIAGRAMS

GoTicket — System Architecture & Design Diagrams

1. System Architecture Overview

GoTicket implements a clean, layered Single Page Application (SPA) architecture that strictly isolates presentation components from domain business logic.

```
[System Diagram / State Flow]
graph TB
    subgraph Client_Browser [Client Browser Runtime (React 19 SPA)]
        subgraph Presentation_Layer [Presentation & Routing Layer]
            App[App.js / Layout.jsx]
            Router[React Router DOM v7]
            Hero[Hero.jsx Search]
            AvailableBuses[AvailableBuses.jsx]
            SelectSeats[SelectSeats.jsx]
            DropPage[DropPage.jsx]
            PaymentPage[PaymentPage.jsx]
            ETicket[ETicket.jsx]
            LiveTracking[LiveTracking.jsx]
            ChatUI[ChatModal.jsx (Tixie UI)]
            AuthModal[AuthModal.jsx]
        end
    end

    subgraph Service_Layer [Domain Business Service Layer]
        travelAgent[travelAgent.js (NLU & FSM)]
        recommendationEngine[recommendationEngine.js (5-Factor Scoring)]
        agentTools[agentTools.js (Tool Facade)]
        busService[busService.js (Query & Latency Mock)]
        seatService[seatService.js (Atomic Seat Validator)]
        bookingService[bookingService.js (Ticket Committer)]
        authService[authService.js (Session Manager)]
        notificationService[notificationService.js (Simulated Alerts)]
    end

    subgraph Storage_Layer [Storage & External Integrations]
        localStorage[(Browser localStorage)]
        mockBuses[(mockBuses.js Master Dataset)]
        OSM[OpenStreetMap Tile Iframe]
        GeoAPI[HTML5 Geolocation API]
    end

    App --> Router
    Router --> Hero
    Router --> AvailableBuses
    Router --> SelectSeats
    Router --> DropPage
    Router --> PaymentPage
    Router --> ETicket
    Router --> LiveTracking
    Router --> AuthModal
    App --> ChatUI

    ChatUI --> travelAgent
    travelAgent --> agentTools
    agentTools --> recommendationEngine
    agentTools --> busService
    agentTools --> seatService
    agentTools --> bookingService
```

```

Hero --> busService
AvailableBuses --> busService
SelectSeats --> seatService
PaymentPage --> bookingService
PaymentPage --> notificationService
AuthModal --> authService
LiveTracking --> OSM
LiveTracking --> GeoAPI
busService --> mockBuses
bookingService --> localStorage
authService --> localStorage

```

2. Conversational Booking Sequence Diagram (Tixie)

[System Diagram / State Flow]

sequenceDiagram

autonumber

actor User

participant Chat as ChatModal.jsx

participant Agent as travelAgent.js (NLU & FSM)

participant Tools as agentTools.js

participant Rec as recommendationEngine.js

participant BusSvc as busService.js

participant SeatSvc as seatService.js

participant BookSvc as bookingService.js

participant PayPage as PaymentPage.jsx

participant LocalStore as localStorage

User->>Chat: "Find buses from Kanpur to Delhi tomorrow evening"

Chat->>Agent: processAgentMessage(query, agentState)

Agent->>Agent: extractTravelParams() -> { source: 'Kanpur', dest: 'Delhi', date: '...', time: '07:00 PM' }

Agent->>Tools: search_buses(params)

Tools->>BusSvc: searchBuses(params)

BusSvc-->>Tools: Returns Matching Bus Array

Tools-->>Agent: Bus Search Results

Agent->>Rec: rankBuses(buses, { preferredTime })

Rec-->>Agent: Returns 5-Factor Ranked Buses & Reasons

Agent-->>Chat: Displays Top Buses + "Why Recommended" Cards & Action Chips

User->>Chat: "Select the recommended one"

Chat->>Agent: processAgentMessage()

Agent->>Tools: check_seat_availability(busId)

Tools->>SeatSvc: getAvailableSeats(busId)

SeatSvc-->>Tools: Available & Occupied Seat Arrays

Tools-->>Agent: Seat Map Data

Agent-->>Chat: Prompts for Seat Numbers (e.g., "S3 and S4")

User->>Chat: "Book S3 and S4"

Chat->>Agent: processAgentMessage()

Agent->>Agent: extractSeatIDs() -> ['S3', 'S4']

Agent->>Tools: hold_select_seats(['S3', 'S4'])

Tools->>SeatSvc: validateAndHoldSeats(['S3', 'S4'])

SeatSvc-->>Tools: { success: true, atomicityHeld: true, totalFare: 1360 }

Tools-->>Agent: Atomic Hold Success

Agent-->>Chat: Prompts: "What is passenger's full name?"

User->>Chat: "Anshika Pandey, anshika@example.com, 9876543210"

Chat->>Agent: processAgentMessage()

Agent->>Agent: Sequential / Single-Sentence Pax Parser

Agent-->>Chat: Renders Transparent Itemized Booking Summary

User->>Chat: "Yes, confirm" (Explicit Confirmation)

Chat->>Agent: processAgentMessage()

```

Agent-->>Tools: prepare_booking(bookingDetails)
Tools-->>BookSvc: preparePendingBooking(bookingDetails)
BookSvc-->>LocalStorage: setItem('pendingBooking', JSON)
BookSvc-->>Tools: { success: true, pendingBooking }
Tools-->>Agent: Booking Staged
Agent-->>Chat: Renders "Proceed to Payment" Action Card
|
User-->>Chat: Clicks "Proceed to Payment (1210)"
Chat-->>PayPage: navigate('/payment', { state: { bookingData } })
User-->>PayPage: Completes Simulated Payment / OTP
PayPage-->>BookSvc: finalizeBooking()
BookSvc-->>LocalStorage: setItem('lastTicket', ticketJSON)
BookSvc-->>LocalStorage: removeItem('pendingBooking')
PayPage-->>User: Renders E-Ticket with QR Code

```

3. Manual Booking Workflow State Diagram

[System Diagram / State Flow]

stateDiagram-v2

```

[*] --> Search_Input: User opens Hero or /seatbooking
Search_Input --> Validating_Query: Enters Source, Destination, Date
Validating_Query --> Search_Input: Error (Same cities / Past date)
Validating_Query --> Available_Buses_View: Valid Query (Passes state to /available-buses)
|
Available_Buses_View --> Available_Buses_View: Filter AM/PM slots
Available_Buses_View --> Select_Seats_View: User clicks Time Slot button
|
Select_Seats_View --> Select_Seats_View: Toggles Available Seats (S1..S40)
Select_Seats_View --> Select_Seats_View: Rejects Sold Seats (S2, S7, S12, S18, S24, S31)
Select_Seats_View --> Drop_Page_View: Clicks "Proceed" with >=1 seat selected
|
Drop_Page_View --> Payment_Wizard: Selects Boarding & Dropping Points
|
state Payment_Wizard {
    [*] --> Summary_Step
    Summary_Step --> Passenger_Details_Step: Clicks "Add Passenger Details"
    Passenger_Details_Step --> Passenger_Details_Step: Fills Name, Email, Aadhaar, Age
    Passenger_Details_Step --> Mobile_OTP_Verification: Clicks "Send OTP"
    Mobile_OTP_Verification --> Passenger_Details_Step: Enters 6-digit Code (mobileVerified=true)
    Passenger_Details_Step --> Payment_Method_Step: Clicks "Proceed to Payment"
    Payment_Method_Step --> Payment_Method_Step: Applies Promo (FIRSTGO / GTWEEKEND)
    Payment_Method_Step --> Payment_Method_Step: Enters UPI ID or Card Details
    Payment_Method_Step --> Booking_Finalized: Clicks "Confirm & Pay"
}
|
Booking_Finalized --> ETicket_View: Commits lastTicket to localStorage
ETicket_View --> [*]: Displays QR Code, Print & Download

```

4. Tixie Finite State Machine (FSM) Transition Model

[System Diagram / State Flow]

stateDiagram-v2

```

[*] --> IDLE: App Launch / Clear
|
IDLE --> COLLECTING_INFORMATION: Incomplete Route/Date Query
IDLE --> WAITING_FOR_BUS_SELECTION: Complete Travel Query
|
COLLECTING_INFORMATION --> COLLECTING_INFORMATION: Missing Source or Destination Prompt
COLLECTING_INFORMATION --> WAITING_FOR_BUS_SELECTION: Route Entities Resolved
|
WAITING_FOR_BUS_SELECTION --> WAITING_FOR_SEAT_SELECTION: Bus Selected ("recommended" / "2nd bus")
WAITING_FOR_BUS_SELECTION --> COLLECTING_INFORMATION: "Modify route / Change date"

```

```
WAITING_FOR_SEAT_SELECTION --> WAITING_FOR_SEAT_SELECTION: Atomic Seat Conflict (Occupied Seat Rejected)
WAITING_FOR_SEAT_SELECTION --> COLLECTING_PASSENGER_INFO: Seats Validated Atomically
WAITING_FOR_SEAT_SELECTION --> BOOKING_SUMMARY: Seats + All Pax Details Provided in Single Turn

COLLECTING_PASSENGER_INFO --> COLLECTING_PASSENGER_INFO: Sequential Missing Prompts (Name -> Email -> Phone)
COLLECTING_PASSENGER_INFO --> BOOKING_SUMMARY: All 3 Pax Fields Satisfied

BOOKING_SUMMARY --> WAITING_FOR_SEAT_SELECTION: "Change seat"
BOOKING_SUMMARY --> COLLECTING_PASSENGER_INFO: "Change name / email / phone"
BOOKING_SUMMARY --> IDLE: "Cancel" / "Abort"
BOOKING_SUMMARY --> PAYMENT_PENDING: Explicit Confirmation ("Yes, confirm" / "Book it")

PAYMENT_PENDING --> CONFIRMED: Payment Finalized on /payment
PAYMENT_PENDING --> IDLE: "Cancel booking"

CONFIRMED --> IDLE: "Book another trip"
CONFIRMED --> [*]: User closes chat
```

5. Live Bus Telemetry & Telematics Architecture

[System Diagram / State Flow]

flowchart LR

subgraph Fleet_Catalog [Fleet Catalog (BUS_FLEET)]

UP32[UP32AB1234: Lucknow ISBT -> Unnao -> Kanpur Central]

DL01[DL01CD5678: Delhi ISBT -> Mathura -> Agra Fort]

MH12[MH12EF9012: Mumbai Dadar -> Lonavala -> Pune Swargate]

end

subgraph Simulation_Engine [Client Simulation Engine (LiveTracking.jsx)]

Timer[3000ms setInterval Timer]

Progress[Progress Incrementer +0.3% per tick]

Interpolate[Linear Interpolator: getBusPosition]

end

subgraph Visualization [Rendering & Distance Layer]

OSM[OpenStreetMap Embed Iframe with bbox]

PinOverlay[Custom CSS Animated Bus Pin Overlay]

GeoAPI[HTML5 Geolocation API]

Haversine[Haversine Formula Distance Engine]

end

Fleet_Catalog --> Timer

Timer --> Progress

Progress --> Interpolate

Interpolate --> OSM

Interpolate --> PinOverlay

GeoAPI --> Haversine

Interpolate --> Haversine

Haversine --> DisplayDist[Displays Live Distance in km]

6. Component Hierarchy & Layout Structure

```
<App>
```

```
  <AuthProvider>
```

```
    <Router>
```

```
      <Layout>
```

```
        <Header />
```

```
          Logo & Laurel Trust Badge
```

```
          Navigation Category Links
```

```
          Auth Status / Avatar / Login Trigger
```

```
        
```

```

    ■■■■ <Outlet> (Dynamic Route Views)
    ■ ■■■■ <Hero /> (Landing Page Search)
    ■ ■■■■ <OffersSection /> (Promo Modal)
    ■ ■■■■ <About /> (Advantages & Counters)
    ■ ■■■■ <BookingSection /> (4-Step Guide)
    ■ ■■■■ <TrackBus /> (Tracking Promo Card)
    ■ ■■■■ <TestimonialsSection /> (Marquee Reviews)
    ■ ■■■■ <FAQ /> (Accordion FAQs)
    ■ ■■■■ <Home /> (Corridor Guide)
    ■ ■■■■ <AvailableBuses /> (Bus Cards & Slots)
    ■ ■■■■ <SelectSeats /> (40-Seat Bus Grid)
    ■ ■■■■ <DropPage /> (Pickup/Drop Radios)
    ■ ■■■■ <PaymentPage /> (Multi-Pax Wizard & OTP)
    ■ ■■■■ <ETicket /> (Pass & QR View)
    ■ ■■■■ <LiveTracking /> (OSM Map Simulation)
    ■ ■■■■ <ContactUs /> (Hotlines & PNR Lookup)
    ■ ■■■■ <Login /> / <Signup /> (Auth Forms)
    ■ ■■■■ Policy Pages (Privacy, Terms, NDA, Reference, Closure, Criteria)
    ■
    ■■■■ <Footer />
    ■ ■■■■ Navigation Links, Helplines & Copyright
    ■
    ■■■■ <ChatbotLauncher />
    ■■■■ <ChatModal /> (Tixie Chat & Action Cards)

```

7. Deployment & Hosting Model

GoTicket is hosted as a high-availability client-side Single Page Application on **Vercel**.

```

[System Diagram / State Flow]
flowchart TD
    Git[Git[GitHub Repository: master branch]] -->|Push / Merge Trigger| VercelCI[Vercel CI/CD Build Pipeline]
    subgraph Build_Step [Build Process]
        NPM_I[npm install dependencies]
        CRA_BUILD[react-scripts build]
        Bundle[Production Webpack Bundle in /build]
    end
    subgraph Edge_Routing [Vercel Edge Rewrite Engine]
        VercelConfig[vercel.json: Rewrites all /* to /index.html]
    end
    subgraph CDN_Distribution [Global Edge Network]
        EdgeCDN[Vercel Global CDN]
        BrowserClient[Client Web Browser]
    end
    Git --> VercelCI
    VercelCI --> NPM_I
    NPM_I --> CRA_BUILD
    CRA_BUILD --> Bundle
    Bundle --> VercelConfig
    VercelConfig --> EdgeCDN
    EdgeCDN --> BrowserClient

```


PART 4: VIVA EXAMINATION GUIDE & Q&A

GoTicket — Comprehensive Viva Examination Guide

This document contains 35+ technical questions and authoritative, technically honest answers tailored for the **B.Tech Major Project Viva Examination and Faculty Evaluation**.

1. General Project & Domain Questions

Q1: What is GoTicket, and what core problem does it solve?

Answer:

GoTicket is a dual-mode Indian intercity bus reservation and fleet tracking platform built with React 19. It addresses the high cognitive load, form fatigue, and rigid filtering of conventional booking platforms by providing two synchronized booking pathways: 1. An autonomous conversational AI travel concierge named **"Tixie"** that guides users through natural language search, deterministic multi-criteria recommendations, atomic seat selection, and safe payment handoff. 2. A streamlined **Manual Booking Web Workflow** featuring 50+ city auto-suggestions, an interactive 40-seat layout, multi-passenger registration with simulated mobile OTPs, promo coupon discounts, and instant printable E-Tickets.

Q2: What makes GoTicket unique compared to existing bus booking platforms?

Answer:

Unlike conventional booking portals that treat chatbots as simple FAQ redirect widgets, GoTicket integrates **Tixie directly into the transactional booking pipeline**. Tixie maintains conversational context, executes tool-based bus queries, calculates 5-factor multi-criteria recommendation scores, performs atomic seat availability validation, enforces a strict explicit confirmation gate, and seamlessly transfers state to the payment checkout flow.

Q3: What is the current architectural maturity of GoTicket?

Answer:

GoTicket is currently implemented as a **fully functional, self-contained client-side Single Page Application (SPA) prototype**. Business logic, state transitions, atomic validations, and mock services run within client-side JavaScript services, with data persistence in browser `localStorage`. External integrations—such as payment gateways, physical GPS telematics, SMS/email dispatches, and cloud LLM APIs—are simulated in the browser for reliable, zero-cost academic demonstration.

2. React 19 & Frontend Architecture

Q4: Why was React chosen as the frontend library for GoTicket?

Answer:

React was selected for its component-based declarative architecture, virtual DOM reconciliation, rich ecosystem, and unidirectional data flow. This modularity allowed a 4-member team to work concurrently on separate components (e.g., `Hero.jsx`, `SelectSeats.jsx`, `PaymentPage.jsx`, and `ChatModal.jsx`) while sharing a unified domain service layer.

Q5: How does GoTicket manage global application state?

Answer:

Global authentication state is managed using the **React Context API** via `AuthContext.jsx` and consumed via the

`useAuth()` custom hook. Journey state (selected bus, departure slot, chosen seats, boarding/dropping points) is passed downstream between routed views using **React Router DOM navigation state** (`navigate(path, { state: { ... } })`), preventing unnecessary global state pollution.

Q6: What React Hooks are utilized across the codebase?

Answer:

* `useState`: For managing local component state (e.g., active chat messages, selected seats, form inputs). * `useEffect`: For side effects such as restoring authentication sessions on mount, initializing bus search queries, and auto-scrolling chat dialogue. * `useRef`: For managing DOM node references (e.g., smooth auto-scrolling in `ChatModal.jsx` and OTP countdown timer intervals in `PaymentPage.jsx`). * `useCallback`: For memoizing event handlers and performance-sensitive calculation functions. * `useLocation` & `useNavigate`: React Router hooks for reading route state and executing client-side navigation.

Q7: Why are CSS Modules used instead of plain global CSS?

Answer:

CSS Modules (e.g., `payment.module.css`, `selectseats.module.css`) automatically generate unique, scoped class names at build time. This guarantees style encapsulation and eliminates accidental CSS namespace collisions across components authored by different team members.

3. JavaScript & Web Standards

Q8: How does GoTicket simulate asynchronous API behavior?

Answer:

Domain services (such as `busService.js` and `agentTools.js`) return native JavaScript `Promise` objects wrapped in `setTimeout` timers (e.g., simulating 500ms network latency). This allows the frontend to implement realistic loading spinners, error boundaries, and asynchronous UI states that mirror a production REST API.

Q9: What are the benefits and limitations of using `localStorage` for data storage?

Answer:

Benefits: Synchronous access, zero configuration, persistent across page reloads, and requires no external database server for running demonstrations. * *Limitations:* * Synchronous I/O can block the main thread if large datasets are parsed; storage capacity is typically limited to 5MB per origin; data is stored in plain text (unencrypted); and data is isolated to a single client browser without multi-user cross-device synchronization.

4. Tixie — Conversational AI Travel Agent

Q10: Is Tixie powered by an external Large Language Model (LLM) like GPT-4 or Gemini?

Answer:

No. Tixie is engineered using a **deterministic client-side Natural Language Understanding (NLU) parser and Finite State Machine (FSM)** implemented in `src/services/travelAgent.js`. This architectural choice guarantees 100% predictable state transitions, zero execution latency, zero API costs, and absolute freedom from LLM hallucinations during transactional booking steps.

Q11: How does Tixie extract travel entities from free-form user messages?

Answer:

Tixie uses deterministic regular expressions and token normalization: * **Cities:** Matches against a curated dictionary of 22 Indian cities across patterns like `from <city> to <city>` and `<city> -> <city>`. * **Dates:** Resolves keywords (`today`,

tomorrow, day after tomorrow, days of the week) to ISO YYYY-MM-DD dates. * **Times:** Identifies explicit times (9 PM, 08:30 AM) and relative time windows (morning, evening, night). * **Seats:** Matches patterns like `\bs[-_s]?d{1,2}\b` and `seats 3 and 4`. * **Passenger Information:** Extracts Name, Email, and 10-digit Indian Mobile number using dedicated regex validators.

Q12: Explain the dialogue states in Tixie's Finite State Machine.

Answer:

1. **IDLE:** Ready for a fresh travel query. 2. **COLLECTING_INFORMATION:** Prompts for missing departure or arrival city. 3. **WAITING_FOR_BUS_SELECTION:** Presents 5-factor ranked buses with action chips. 4. **WAITING_FOR_SEAT_SELECTION:** Prompts for seat numbers and executes atomic validation. 5. **COLLECTING_PASSENGER_INFO:** Collects Full Name, Email, and Phone number sequentially or in a single turn. 6. **BOOKING_SUMMARY:** Displays an itemized, masked summary and awaits user decision. 7. **PAYMENT_PENDING:** Stages the booking in `localStorage['pendingBooking']` and renders the payment handoff action card. 8. **BOOKING:** Transient lock preventing duplicate submissions. 9. **CONFIRMED:** Post-booking state providing E-Ticket links.

Q13: What is the "Strict Explicit Confirmation Gate" in Tixie, and why is it essential?

Answer:

The Explicit Confirmation Gate is a human-in-the-loop safety mechanism. Tixie **never finalizes a booking or initiates payment handoff merely because a user discusses travel**. When in **BOOKING_SUMMARY** state, Tixie strictly requires unambiguous confirmation phrases (e.g., "Yes, confirm", "Confirm booking", "Book it"). Non-committal phrases ("maybe", "how much is it?") or questions keep the agent in summary state, preventing accidental commitments.

Q14: How does Tixie handle inline user modifications?

Answer:

If a user is at the final booking summary and says "change seat to S5" or "change email to user@test.com", Tixie does not reset the entire conversation. Instead, it transitions back to the specific sub-state (**WAITING_FOR_SEAT_SELECTION** or **COLLECTING_PASSENGER_INFO**), updates the target parameter, and rebuilds the summary immediately.

5. Recommendation Engine

Q15: How does GoTicket rank available buses?

Answer:

Buses are ranked deterministically by `src/services/recommendationEngine.js` using a **5-Factor Multi-Criteria Decision Analysis (MCDA) weighted scoring formula**:
$$\text{Total Score} = (S_{\text{time}} \times 0.30) + (S_{\text{price}} \times 0.25) + (S_{\text{duration}} \times 0.20) + (S_{\text{seats}} \times 0.15) + (S_{\text{rating}} \times 0.10)$$

Q16: Why are these specific weights assigned?

Answer:

In Indian intercity bus transit, commuter research shows that **departure time convenience (30%)** and **ticket price (25%)** are the primary decision drivers. **Journey duration (20%)** and **seat availability (15%)** reflect comfort and booking feasibility, while **operator rating (10%)** provides quality assurance without overpowering schedule compatibility.

Q17: Is this recommendation engine an example of Machine Learning model training?

Answer:

No. It is a **deterministic multi-criteria scoring algorithm (expert system)** based on decision utility theory. It does not require supervised training or gradient descent, making it fast, transparent, mathematically explainable, and fully auditable.

6. Seat Management & Atomic Validation

Q18: What is "Atomic Seat Validation", and why is it implemented?

Answer:

Atomic Seat Validation (`seatService.validateAndHoldSeats`) enforces the all-or-nothing transactional property. If a passenger requests multiple seats (e.g., `["S3", "S7"]`) and even **one** requested seat is occupied (S7), the **entire selection fails atomically**. No partial seats (S3) are reserved. This prevents fragmented bookings and ensures multi-traveler groups sit together or receive immediate feedback.

Q19: What is the seat layout configuration in GoTicket?

Answer:

The layout consists of **40 seats (S1 to S40)** arranged across 8 rows $\times$ 5 columns (2 Left Seats + Central Aisle + 2 Right Seats). Default occupied seats (`['S2', 'S7', 'S12', 'S18', 'S24', 'S31']`) are locked in disabled gray on the UI.

7. Payment Flow & Handoff

Q20: Does Tixie process payments directly inside the chat window?

Answer:

No. To comply with secure payment design principles, Tixie stages the booking context into `localStorage['pendingBooking']` and renders a secure **"Proceed to Payment"** action card. Clicking this transfers control to the dedicated `PaymentPage.jsx` wizard where passenger verification, promotional coupons, and payment methods are executed.

Q21: How is payment checkout simulated?

Answer:

`PaymentPage.jsx` supports UPI ID verification (checks for @ symbol) and Card input validation (16 digits, 3-digit CVV, valid expiry). When "Confirm & Pay" is clicked, `finalizeBooking()` commits the finalized ticket to `localStorage['lastTicket']`, purges `pendingBooking`, and displays the confirmation screen.

8. Live Bus Tracking & OpenStreetMap

Q22: How does live bus tracking work in GoTicket?

Answer:

Tracking is implemented in `src/pages/aboutgoticket/LiveTracking.jsx`. It maintains a catalog of predefined route waypoints (`BUS_FLEET`). A `3000ms setInterval` timer increments progress by `+0.3%` per tick and linearly interpolates the vehicle's latitude and longitude between consecutive waypoints.

Q23: How is OpenStreetMap integrated?

Answer:

GoTicket embeds an OpenStreetMap export iframe centered around a dynamic bounding box computed from the interpolated vehicle coordinates:

```
https://www.openstreetmap.org/export/embed.html?bbox={lng-0.15},{lat-0.15},{lng+0.15},{lat+0.15}&layer=mapnik&marker={lat
```

A custom animated CSS bus pin overlay is rendered above the iframe to visualize real-time transit.

Q24: How does the application calculate distance between the user and the bus?

Answer:

The application requests device coordinates via the browser's `navigator.geolocation.getCurrentPosition` API and

computes the great-circle distance to the moving bus using the **Haversine Formula**: $d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_1)\cos(\text{lat}_2)\sin^2\left(\frac{\Delta\text{lng}}{2}\right)}\right)$

9. Security & Production Roadmap

Q25: What are the security limitations of the current prototype?

Answer:

1. Passwords in `localStorage['registeredUsers']` are stored unhashed in plain text. 2. Authentication tokens are client-generated mock strings rather than cryptographically signed JWTs. 3. Sensitive passenger information (e.g., masked Aadhaar) is held in client storage.

Q26: How will GoTicket transition to a secure production architecture?

Answer:

1. Connect to a **FastAPI or Express REST backend** running HTTPS. 2. Implement **bcrypt password hashing** and **asymmetric JWT authentication** with HTTP-only secure cookies. 3. Integrate **Razorpay / Stripe Sandbox** with server-side webhook signature verification. 4. Replace `mockBuses.js` with a **PostgreSQL database** managed via SQLAlchemy / Prisma ORM. 5. Ingest live GPS telematics from vehicle OBD-II devices over **WebSockets or MQTT**.

PART 5: PROJECT DEMONSTRATION SCRIPT

GoTicket — Project Demonstration & Presentation Script

This script provides a structured **7 to 10 minute live presentation and demonstration walkthrough** for the B.Tech AI & ML Major Project Evaluation and Viva Examination.

Presentation Overview & Timing

Phase	Section Title	Allocated Time	Presenter Focus
Phase 1	Project Introduction & Core Problem	1.0 Minute	Motivation, dual-mode concept, tech stack
Phase 2	End-to-End Manual Booking Workflow	2.5 Minutes	Search, seat map, OTP verify, coupons, E-Ticket
Phase 3	Live Bus Tracking & Telemetry	1.5 Minutes	OpenStreetMap, waypoint simulation, Haversine
Phase 4	Autonomous AI Concierge ("Tixie")	3.5 Minutes	NLU, 5-factor scoring, atomic seats, confirmation gate
Phase 5	Architecture Summary & Conclusion	1.0 Minute	Service decoupling, data schemas, Q&A handoff

Phase 1: Introduction & Project Overview (0:00 – 1:00)

Spoken Dialogue (Presenter 1):

"Good morning respected evaluators and faculty members. Today, our team is proud to present GoTicket, an AI-Agent-Based Intercity Bus Booking Platform developed as our major B.Tech AI & ML capstone project.

In India, intercity bus travel carries millions daily, yet online booking portals often suffer from form fatigue, rigid filters, and high cognitive load when comparing disparate buses. GoTicket resolves this by introducing a dual-mode platform: combining a streamlined manual booking flow with Tixie, an autonomous conversational AI travel concierge.

The entire frontend is built on React 19 and React Router DOM v7, featuring a decoupled domain service layer, deterministic multi-criteria recommendations, atomic seat validation, and simulated fleet telemetry."

Phase 2: Manual Booking Workflow (1:00 – 3:30)

Action 1: Landing Page & City Search

- UI Action:** Open homepage (<http://localhost:3000>). Click the origin city input, type "Kanpur", choose destination "Delhi", and select tomorrow's date. Click **"Search Buses"**.

Spoken Dialogue:

"Starting with our manual workflow, our Hero search bar supports auto-suggestions across 50+ Indian cities and validates against identical source/destination or past travel dates. Submitting the search navigates to the Available Buses view with simulated async latency."

Action 2: Bus Results & Time Slot Selection

- **UI Action:** View bus cards. Highlight operator ratings, AC bus types, and amenities. Click on the 08:30 PM (■680) time slot button on **GoRide Express**.

Spoken Dialogue:

"Here, search results present comprehensive bus information—including operator ratings, amenities like Wi-Fi and charging points, and multiple departure time slots. Selecting a time slot transitions us to the interactive seat map."

Action 3: Interactive Seat Map & Sold Seat Locks

- **UI Action:** Show 40-seat layout. Attempt clicking gray sold seat S7 (disabled). Select S3 and S4 (turn green). Total fare updates to ■1,360. Click **"Proceed"**.

Spoken Dialogue:

"Our interactive seat map renders an 8-row by 5-column bus layout with a central aisle. Notice that occupied seats like S7 are locked in disabled gray. Selecting available seats S3 and S4 dynamically computes our fare and passes state downstream."

Action 4: Boarding & Dropping Points

- **UI Action:** Select Boarding Point (Jhatkari Bus Station, Kanpur) and Dropping Point (Transport Nagar Metro, Lucknow). Click **"Proceed to Payment"**.

Spoken Dialogue:

"Next, the traveler selects route-specific boarding and dropping checkpoints before proceeding to the checkout wizard."

Action 5: Multi-Passenger Details, OTP Verification & Coupons

- **UI Action:** Fill passenger name ("Rahul Sharma"), 10-digit mobile ("9876543210"), email, and Aadhaar. Click **"Send OTP"** (alert appears showing 6-digit demo OTP). Enter OTP and click **"Verify"** (shows green verified badge). Apply coupon code **FIRSTGO** (fare reduces by ■150).

Spoken Dialogue:

"On the Payment page, we support dynamic multi-passenger forms. We simulate mobile OTP verification where a 6-digit code verifies the traveler's phone number. We can also apply discount vouchers—like FIRSTGO for ■150 off—and select UPI or Card checkout."

Action 6: Confirmation & Printable E-Ticket

- **UI Action:** Select UPI (rahul@okaxis) and click **"Confirm & Pay"**. The E-Ticket view renders with reference ID (e.g., GT7AX9KP), QR code, passenger details, and a Print/Download button.

Spoken Dialogue:

"Upon payment confirmation, bookingService commits the finalized ticket object to browser localStorage and displays a printable digital E-Ticket complete with QR verification code and reference ID."

Phase 3: Live Bus Tracking & Telemetry (3:30 – 5:00)

Action: Open Live Tracking Page

- **UI Action:** Navigate to `/livettracking`. Enter bus number `UP32AB1234` (or click the quick route button for Lucknow o Kanpur).

Spoken Dialogue (Presenter 2):

"Now, let us examine our Live Bus Tracking module. GoTicket embeds an OpenStreetMap iframe bounded dynamically around the vehicle's coordinates.

In this demonstration mode, vehicle movement is simulated by a 3-second timer that linearly interpolates the bus position between predefined route waypoints—from Lucknow ISBT through Unnao to Kanpur Central.

Furthermore, by invoking the browser's HTML5 Geolocation API, the platform uses the mathematical Haversine formula to calculate the real-time great-circle distance between the user's physical device and the moving bus in kilometers."

Phase 4: Conversational AI Travel Concierge — Tixie (5:00 – 8:30)

Action 1: Open Tixie & Natural Language Route Search

- **UI Action:** Click the floating **Tixie avatar** in the bottom right corner. Type:
`Find buses from Kanpur to Delhi tomorrow evening`

Spoken Dialogue (Presenter 3):

"Now we introduce Tixie, our autonomous conversational AI travel concierge. When we send a natural language travel request, Tixie's regex NLU parser extracts the departure city Kanpur, destination Delhi, tomorrow's date, and the evening departure window (07:00 PM)."

Action 2: Deterministic 5-Factor Recommendation

- **UI Action:** Point to the ranked bus output and the "Why GoRide Express?" explanation card.

Spoken Dialogue:

"Notice how Tixie ranks the buses using our deterministic 5-factor scoring engine: weighting Time Match (30%), Price (25%), Duration (20%), Seat Availability (15%), and Customer Rating (10%). Tixie explicitly explains why the top bus is recommended."

Action 3: Bus Selection & Atomic Seat Conflict Test

- **UI Action:** Click chip "**Select the recommended one**". Tixie shows available seats. Type:
`Book S5 and S7`

Spoken Dialogue:

"Let's select the recommended bus. Now let's test our Atomic Seat Validation. We deliberately request seat S5 along with seat S7, which is already occupied in our dataset."

- **Expected UI Output:** Tixie replies: ■■ Seat Validation Notice: S7 is unavailable, but S5 is available. No seats were committed due to atomic availability checks.

Spoken Dialogue:

"Because seat S7 is occupied, our atomic validator rejects the entire request immediately, preventing partial seat allocation. Now let's request available seats: [Book S3 and S4](#)."

Action 4: Passenger Information Collection

- **UI Action:** Type:

[Book S3 and S4](#)

Then, when prompted, type in a single sentence:

[Name is Anshika Pandey, email anshika@example.com, mobile 9876543210](#)

Spoken Dialogue:

"Tixie supports both step-by-step and single-sentence passenger extraction. Notice that it extracts the full name, email, and 10-digit mobile number simultaneously and immediately compiles the transparent Booking Summary."

Action 5: Strict Explicit Confirmation Gate

- **UI Action:** Type:

[Maybe, how much is the total?](#)

- **Expected UI Output:** Tixie re-prompts for explicit confirmation without creating a booking.

Spoken Dialogue:

"Here we demonstrate our Strict Explicit Confirmation Gate. When the user gives a non-committal response like 'Maybe', Tixie does NOT book the ticket. It preserves state and prompts for explicit approval."

- **UI Action:** Type:

[Yes, confirm](#)

- **Expected UI Output:** Tixie transitions to [PAYMENT_PENDING](#) and renders the **"Proceed to Payment (■1210)"** action card.

Spoken Dialogue:

"Once unambiguous confirmation is given, Tixie stages the booking into `localStorage['pendingBooking']` and hands off control to the Payment page to finalize checkout securely."

Phase 5: Architecture Summary & Conclusion (8:30 – 10:00)

Spoken Dialogue (Presenter 4):

"To summarize our architectural achievements: 1. Decoupled Service Layer: UI components remain purely presentational, interfacing with services like `travelAgent.js`, `recommendationEngine.js`, and `seatService.js`. 2. Transactional Integrity: Enforced via atomic seat locking and explicit confirmation gates. 3. Production Readiness: The service layer contracts are structured such that swapping mock functions for REST API calls to a FastAPI or Express backend requires zero UI code modifications.*

Thank you for your time and attention. We are now ready to take your questions."

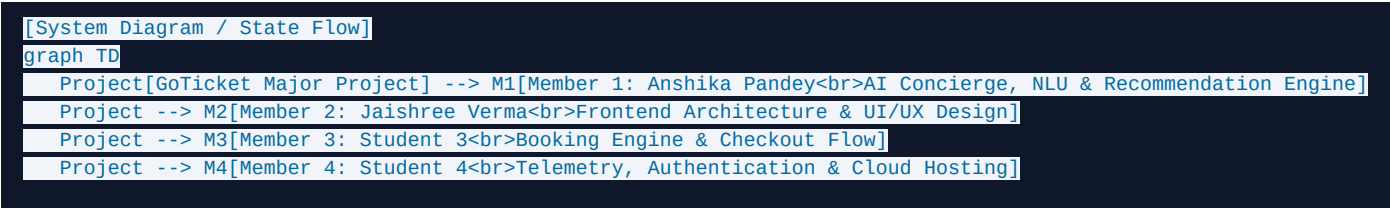
PART 6: TEAM CONTRIBUTION & OWNERSHIP MATRIX

GoTicket — Team Workload & Module Contribution Matrix

Final Year B.Tech AI & ML Major Capstone Project

This document outlines the division of engineering responsibilities, module ownership, and assigned source files across the 4 final-year B.Tech student contributors.

1. Executive Summary of Team Responsibilities



2. Detailed Member Contribution Breakdown

Member 1: AI Travel Concierge & Conversational NLU Lead

- **Name:** Anshika Pandey
- **Roll Number:** [Roll Number Placeholder 1]
- **Email:** 2k23.psitaимl2311222@gmail.com
- **Core Module:** Conversational Agent ("Tixie"), NLU Entity Extraction, Multi-Turn FSM, Recommendation Scoring, and Atomic Agent Tools.
- **Specific Responsibilities:** 1. Engineered the 1,130-line conversational orchestrator in `src/services/travelAgent.js`. 2. Implemented deterministic regular expressions for city matching (22 Indian cities), relative dates (`today`, `tomorrow`, days of the week), and time windows. 3. Built the multi-turn Finite State Machine (FSM) handling 9 distinct dialogue states (`IDLE`, `COLLECTING_INFORMATION`, `WAITING_FOR_BUS_SELECTION`, `WAITING_FOR_SEAT_SELECTION`, `COLLECTING_PASSENGER_INFO`, `BOOKING_SUMMARY`, `PAYMENT_PENDING`, `BOOKING`, `CONFIRMED`). 4. Designed the 5-factor multi-criteria bus recommendation scoring algorithm in `src/services/recommendationEngine.js`. 5. Built the tool abstraction facade in `src/services/agentTools.js` integrating seat checking, bus details, and pending booking preparation. 6. Implemented the Strict Explicit Confirmation Gate and in-flight duplicate submission guard.
- **Primary Source Files:**
 - `src/services/travelAgent.js`
 - `src/services/recommendationEngine.js`
 - `src/services/agentTools.js`
 - `src/components/Chatbot/ChatModal.jsx`
 - `src/components/Chatbot/ChatbotLauncher.jsx`
 - `src/components/Chatbot/Chatbot.css`

Member 2: Frontend Architecture & UI/UX Design Lead

- **Name:** Jaishree Verma
- **Roll Number:** [Roll Number Placeholder 2]
- **Email:** jaishree.verma.work@gmail.com
- **Core Module:** Presentation Layer Architecture, Master Layout, Responsive CSS Modules, Landing Page Sections, and Visual Seat Map.
- **Specific Responsibilities:** 1. Architected the master single-page application layout ([Layout.jsx](#), [Header.jsx](#), [Footer.jsx](#)). 2. Developed the Hero search interface with dynamic auto-suggestions for 50+ Indian cities and date validation ([Hero.jsx](#)). 3. Created the 40-seat interactive bus grid layout with central aisle spacing, occupied seat styling, and dynamic seat selection toggles ([SelectSeats.jsx](#)). 4. Designed all component-level and page-level CSS Modules ([src/styles/*.module.css](#) and [src/stylespages/*.module.css](#)), establishing design tokens and dark/light color schemes. 5. Built interactive landing page sections: Animated Counters ([About.jsx](#)), 4-Step Booking Guide ([BookingSection.jsx](#)), Discount Gift Modal ([OffersSection.jsx](#)), Marquee Testimonials Carousel ([TestimonialsSection.jsx](#)), and Accordion FAQs ([FAQ.jsx](#)).
- **Primary Source Files:**
 - [src/components/Hero.jsx](#)
 - [src/components/Header.jsx](#)
 - [src/components/Footer.jsx](#)
 - [src/components/Layout.jsx](#)
 - [src/components/About.jsx](#)
 - [src/components/BookingSection.jsx](#)
 - [src/components/OffersSection.jsx](#)
 - [src/components/TestimonialsSection.jsx](#)
 - [src/components/FAQ.jsx](#)
 - [src/pages/aboutgoticket/SelectSeats.jsx](#)
 - [src/styles/*.module.css](#) & [src/stylespages/*.module.css](#)

Member 3: Booking Engine, Checkout & E-Ticket Lead

- **Name:** [Student 3 Name Placeholder]
- **Roll Number:** [Roll Number Placeholder 3]
- **Email:** [Student 3 Email Placeholder]
- **Core Module:** Search Filtering, Seat Service Logic, Boarding/Dropping Selection, Multi-Passenger Wizard, OTP Simulation, and E-Ticket Generation.
- **Specific Responsibilities:** 1. Implemented the bus schedule search and time slot selection view with AM/PM filters ([AvailableBuses.jsx](#)). 2. Built the boarding and dropping point radio selector with popular checkpoint highlights ([DropPage.jsx](#)). 3. Created the 4-step checkout wizard in [PaymentPage.jsx](#) supporting dynamic multi-passenger forms, simulated 6-digit mobile OTP verification, Aadhaar format validation, and promotional coupon discounts ([FIRSTGO](#), [GTWEEKEND](#), [UPIPAY](#)). 4. Engineered the digital E-Ticket view ([ETicket.jsx](#)) with QR code generation, ticket reference ID ([GTXXXXXX](#)), passenger details, and printable layout. 5. Developed [bookingService.js](#) for generating unique ticket IDs, staging pending bookings, and committing finalized tickets to [localStorage](#). 6. Implemented atomic seat validation in [seatService.js](#) ([validateAndHoldSeats](#)).
- **Primary Source Files:**

- `src/pages/aboutgoticket/AvailableBuses.jsx`
- `src/pages/aboutgoticket/DropPage.jsx`
- `src/pages/aboutgoticket/PaymentPage.jsx`
- `src/pages/aboutgoticket/ETicket.jsx`
- `src/services/bookingService.js`
- `src/services/seatService.js`
- `src/services/busService.js`
- `src/data/mockBuses.js`

Member 4: Telemetry, Authentication & Cloud Hosting Lead

- **Name:** [Student 4 Name Placeholder]
- **Roll Number:** [Roll Number Placeholder 4]
- **Email:** [Student 4 Email Placeholder]
- **Core Module:** Live Bus Telemetry, Geolocation Calculations, Authentication Context, Session Storage, Notification Logger, and Cloud Deployment.
- **Specific Responsibilities:** 1. Developed the live bus tracking simulator with OpenStreetMap iframe embedding, waypoint linear interpolation, and simulated 3-second timer updates (`LiveTracking.jsx`, `TrackBus.jsx`). 2. Implemented the mathematical **Haversine formula** to calculate great-circle distances between the user's browser location (via HTML5 Geolocation API) and the moving bus. 3. Built the global authentication state manager using React Context (`AuthContext.jsx`) and route guard (`ProtectedRoute.jsx`). 4. Developed `authService.js` with demo credentials support (`demo@goticket.in` / `demo123`), user registration, and `localStorage` session persistence. 5. Created `notificationService.js` to format and log simulated booking confirmation emails and SMS alerts with masked recipient details. 6. Configured Vercel single-page application deployment (`vercel.json`), environment flags (`.env`), and GitHub repository workflows.
- **Primary Source Files:**
 - `src/pages/aboutgoticket/LiveTracking.jsx`
 - `src/components/TrackBus.jsx`
 - `src/context/AuthContext.jsx`
 - `src/services/authService.js`
 - `src/services/notificationService.js`
 - `src/components/AuthModal.jsx`
 - `src/components/auth/ProtectedRoute.jsx`
 - `src/pages/auth/Login.jsx`
 - `src/pages/auth/Signup.jsx`
 - `vercel.json` & `.env`

3. Team Ownership Matrix by File

Source File / Service	Primary Owner	Secondary Reviewer	Module Category
<code>src/services/travelAgent.js</code>	Anshika Pandey	Jaishree Verma	AI Agent & NLU

<code>src/services/recommendationEngine.js</code>	Anshika Pandey	[Student 3]	Scoring Algorithm
<code>src/services/agentTools.js</code>	Anshika Pandey	[Student 3]	Agent Tool Facade
<code>src/components/Chatbot/*</code>	Anshika Pandey	Jaishree Verma	Conversational UI
<code>src/components/Hero.jsx</code>	Jaishree Verma	[Student 3]	Frontend Search UI
<code>src/components/Header.jsx & Footer.jsx</code>	Jaishree Verma	[Student 4]	Master Navigation
<code>src/pages/aboutgoticket/SeatsPage.jsx</code>	Jaishree Verma	[Student 3]	Visual Seat Grid
<code>src/styles/* & src/stylespages/*</code>	Jaishree Verma	All Members	CSS Modules & Styling
<code>src/pages/aboutgoticket/AvailableRoutes.jsx</code>	[Student 3]	Jaishree Verma	Search Results
<code>src/pages/aboutgoticket/DropOffPage.jsx</code>	[Student 3]	Jaishree Verma	Route Checkpoints
<code>src/pages/aboutgoticket/PaymentPage.jsx</code>	[Student 3]	Anshika Pandey	Checkout & OTP
<code>src/pages/aboutgoticket/ETicket.jsx</code>	[Student 3]	[Student 4]	E-Ticket Pass
<code>src/services/bookingService.js</code>	[Student 3]	Anshika Pandey	Ticket Committer
<code>src/services/seatService.js</code>	[Student 3]	Anshika Pandey	Atomic Seat Logic
<code>src/pages/aboutgoticket/LiveTracking.jsx</code>	[Student 4]	Jaishree Verma	Telemetry & Maps
<code>src/context/AuthContext.jsx</code>	[Student 4]	[Student 3]	Authentication State
<code>src/services/authService.js</code>	[Student 4]	[Student 3]	Session Persistence
<code>src/services/notificationService.js</code>	[Student 4]	Anshika Pandey	Simulated Alerts
<code>vercel.json & package.json</code>	[Student 4]	All Members	Cloud Deployment

